

A Reliability-Graded Evaluation of Set Transformer Interaction Modeling for Scent Prediction in Two-Molecule Fragrance Blends

Sofia Gabrielle I. Belarmino

Bachelor of Science in Computer Science

Lester Liby V. Ojeñar

Bachelor of Science in Computer Science

Senior Thesis submitted to the faculty of the
Department of Computer Science
College of Computer Studies, Ateneo de Naga University
in partial fulfillment of the requirements for their respective
Bachelor of Science degrees

Project Advisor: Ian Peter L. Lastimosa

Jelly P. Aureus, MS

Jerome G. Flores

Jasper V. Gomez

September 2026

Naga City, Philippines

Keywords: graph neural networks, multi-label classification, set learning

Copyright 2026, Sofia Gabrielle I. Belarmino and Lester Liby V. Ojeñar

The Senior Project entitled

**A Reliability-Graded Evaluation of Set Transformer Interaction
Modeling for Scent Prediction in Two-Molecule Fragrance Blends**

developed by

Sofia Gabrielle I. Belarmino

Bachelor of Science in Computer Science

Lester Liby V. Ojeñar

Bachelor of Science in Computer Science

and submitted in partial fulfillment of the requirements of their respective Bachelor of Science degrees
has been rigorously examined and recommended for approval and acceptance.

Jelly P. Aureus, MS

Panel Member

Date signed: _____

Jerome G. Flores

Panel Member

Date signed: _____

Jasper V. Gomez

Panel Member

Date signed: _____

Ian Peter L. Lastimosa

Project Advisor

Date signed: _____

The Senior Project entitled

**A Reliability-Graded Evaluation of Set Transformer Interaction
Modeling for Scent Prediction in Two-Molecule Fragrance Blends**

developed by

Sofia Gabrielle I. Belarmino

Bachelor of Science in Computer Science

Lester Liby V. Ojeñar

Bachelor of Science in Computer Science

and submitted in partial fulfillment of the requirements of their respective Bachelor of Science degrees is hereby approved and accepted by the Department of Computer Science, College of Computer Studies, Ateneo de Naga University.

Jelly P. Aureus, MS

Chair, Department of Computer Science

Date signed: _____

Marianne A. Tolentino, MS

Dean, College of Computer Studies

Date signed: _____

Declaration of Original Work

We declare that the Senior Project entitled

A Reliability-Graded Evaluation of Set Transformer Interaction Modeling for Scent Prediction in Two-Molecule Fragrance Blends

which we submitted to the faculty of the

Department of Computer Science, Ateneo de Naga University

is our own work. To the best of our knowledge, it does not contain materials published or written by another person, except where due citation and acknowledgement is made in our senior project documentation. The contributions of other people whom we worked with to complete this senior project are explicitly cited and acknowledged in our senior project documentation.

We also declare that the intellectual content of this senior project is the product of our own work. We conceptualized, designed, encoded, and debugged the source code of the core programs in our senior project. The source code of third party APIs and library functions used in our program are explicitly cited and acknowledged in our senior project documentation. Also duly acknowledged are the assistance of others in minor details of editing and reproduction of the documentation.

In our honor, we declare that we did not pass off as our own the work done by another person. We are the only persons who encoded the source code of our software. We understand that we may get a failing mark if the source code of our program is in fact the work of another person.

Sofia Gabrielle I. Belarmino

4 - Bachelor of Science in Computer Science

Lester Liby V. Ojeñar

4 - Bachelor of Science in Computer Science

This declaration is witnessed by:

Ian Peter L. Lastimosa

Project Advisor

A Reliability-Graded Evaluation of Set Transformer Interaction Modeling for Scent Prediction in Two-Molecule Fragrance Blends

by

Sofia Gabrielle I. Belarmino and Lester Liby V. Ojeñar

Project Advisor: Ian Peter L. Lastimosa

Department of Computer Science

ABSTRACT

Predicting the scent of a blend of two aroma chemicals is harder than predicting the scent of one, because a blend does not always smell like the sum of its parts. When graph neural networks are extended from single molecules to blends, they are usually combined by averaging, which assumes the two molecules do not affect each other. This study evaluates a Set Transformer interaction layer that lets two molecule representations adjust to one another before being combined, and compares it against averaging and a concatenation baseline reproduced from an existing published pairwise model. All three models share the same graph attention network encoder and predict a set of scent descriptors for each blend. Because the blend labels used for training and evaluation come from a public dataset compiled from catalog recommendations rather than a controlled sensory panel, their reliability is measured before the models are compared: each blend descriptor is graded as Gold, Silver, or Bronze, based on reciprocal agreement between the two catalog listings of a pair and cross-catalog agreement between GoodScents and Leffingwell on the parent molecules’ descriptors. Within a leakage-resistant test split built from UMAP and K-means clustering, the three models are compared on the high-confidence (Gold-tier) blends using macro F1, micro F1, PR-AUC, and Hamming loss, with paired bootstrap confidence intervals. The comparison is examined separately for blends that are additive and those where a scent descriptor is lost or gained relative to the two parent molecules, and its sensitivity to label reliability is tested by training on Gold-tier blends alone versus all tiers, and under label noise injected at increasing levels.

TABLE OF CONTENTS

1	Introduction	1
1.1	Project Context	2
1.2	Purpose and Description	3
1.3	Objectives	5
1.4	Scope and Limitations	5
1.5	Definition of Terms	7
2	Methodology	9
2.1	Research Design	9
2.2	Data Acquisition and Preparation	10
2.2.1	Data Sources and Provenance	11
2.2.2	Ground Truth and Its Limits	12
2.2.3	Target Vocabulary	12
2.2.4	Label Families (Reporting Only)	13
2.3	Label Reliability Mask	14
2.3.1	Consistency Evidence	14
2.3.2	Eligibility Mask	14
2.3.3	Experimental Label Conditions	15
2.4	Model Architecture	16
2.4.1	Shared Graph Encoder	16
2.4.2	Aggregation Variants	17
2.4.3	Output Head and Loss	19

2.5	Component-Disjoint Partitioning and Graph Carving	19
2.5.1	Scaffold Split Decay	19
2.5.2	Cluster-Based Partitions	20
2.5.3	The Graph-Carving Protocol	21
2.5.4	Training-Only Vocabulary Filter	21
2.6	Experiments	22
2.6.1	Experiments A and B: Main Comparison (RQ1)	22
2.6.2	Experiment C: Controlled Label Corruption (RQ3)	22
2.6.3	Blend-Type Analysis (RQ2)	23
2.7	Evaluation Metrics	23
2.7.1	Multi-Label Metrics	23
2.7.2	Masked Metrics	24
2.8	Statistical Comparison	25
2.9	Reporting Conventions	26
2.9.1	The Principal Odor Map	27
2.9.2	Limitations of Single-Molecule Models	28
2.10	Psychophysics of Odor Mixtures	28
2.10.1	Scent Blending Mechanics	28
2.10.2	Evidence That Averaging May Be Sufficient	29
2.11	Machine Learning on Sets and Chemical Mixtures	30
2.11.1	Permutation Invariance on Sets	30
2.11.2	Precedents in Mixture Modeling	31
2.12	Olfactory Datasets and Label Reliability	33
2.12.1	Reference Catalogs	33
2.12.2	The Blended-Pair Dataset	33
2.12.3	Human-Rated Datasets	34
2.12.4	Absence of a Standard Vocabulary	34
2.12.5	Noisy and Imbalanced Labels in Multi-Label Learning	34
2.13	Out-of-Distribution Splitting Strategies	35
2.14	Evaluating and Comparing Models Under Label Sparsity	36
2.15	Synthesis and Research Gap	37

2.15.1	Synthesis of Literature	37
2.15.2	Statement of the Research Gap	38
2.15.3	Positioning of the Present Study	38
3	Theoretical and Conceptual Framework	39
3.1	Theoretical Framework	39
3.1.1	Graph-Theoretic Molecular Representation	39
3.1.2	Foundations of Graph Neural Networks and Message Passing	42
3.1.3	Deep Learning Theory: Graph Attention Networks (GAT)	43
3.1.4	Set Invariance and Set Transformers	44
3.1.5	Readout Expressivity Limits and Symmetry Collapse	45
3.1.6	Label Reliability: Reciprocal Agreement, Cohen’s Kappa, and Noisy Labels	46
3.2	Conceptual Framework	47
3.2.1	Input Phase	48
3.2.2	Process Phase 1: Data Preparation and Graph Carving	48
3.2.3	Process Phase 2: Encoding and Interaction	49
3.2.4	Process Phase 3: Classification and Optimization	49
3.2.5	Output Phase	50
3.3	Technical Background	50
3.3.1	Cheminformatics Standard (RDKit): SMILES Parsing and Sanitization	50
3.3.2	Deep Learning Stack: Sparse Graph Convolutions	51
3.3.3	Multi-Label Formulation: Sigmoids and Sparsity Mitigation	52
4	Methodology	54
4.1	Research Design	54
4.2	Data Acquisition and Preparation	55
4.2.1	Data Sources and Provenance	56
4.2.2	Ground Truth and Its Limits	57
4.2.3	Target Vocabulary	57
4.2.4	Label Families (Reporting Only)	58
4.3	Label Reliability Mask	59
4.3.1	Consistency Evidence	59

4.3.2	Eligibility Mask	59
4.3.3	Experimental Label Conditions	60
4.4	Model Architecture	61
4.4.1	Shared Graph Encoder	61
4.4.2	Aggregation Variants	62
4.4.3	Output Head and Loss	64
4.5	Component-Disjoint Partitioning and Graph Carving	64
4.5.1	Scaffold Split Decay	64
4.5.2	Cluster-Based Partitions	65
4.5.3	The Graph-Carving Protocol	66
4.5.4	Training-Only Vocabulary Filter	66
4.6	Experiments	67
4.6.1	Experiments A and B: Main Comparison (RQ1)	67
4.6.2	Experiment C: Controlled Label Corruption (RQ3)	67
4.6.3	Blend-Type Analysis (RQ2)	68
4.7	Evaluation Metrics	68
4.7.1	Multi-Label Metrics	68
4.7.2	Masked Metrics	69
4.8	Statistical Comparison	70
4.9	Reporting Conventions	71
A	Chemical Feature Vectors	72
A.1	Symmetry Collapse: Representation-Theoretic Derivation	74
A.2	Dataset Profiles and Canonical Vocabulary	75
A.2.1	Single-Molecule Harmonized Dataset Sample	75
A.2.2	Pairwise Blend Mixture Sample	75
A.2.3	Standardized 104-Odor Descriptor Vocabulary Gate	75

LIST OF FIGURES

3.1 Input-Process-Output (IPO) Model. 47

LIST OF TABLES

2.1	Varied factors and matched controls.	10
2.2	Eligibility of a blend–descriptor cell in the Gold-only treatment.	15
2.3	The three aggregation variants.	17
4.1	Varied factors and matched controls.	55
4.2	Eligibility of a blend–descriptor cell in the Gold-only treatment.	60
4.3	The three aggregation variants.	62
A.1	Specification of the 46-Dimensional Atom (Node) Feature Vector (X_i).	73
A.2	Specification of the 10-Dimensional Bond (Edge) Feature Vector (E_{ij}).	73
A.3	Representative sample of harmonized single odorants.	75
A.4	Sample pairwise odor mixtures from the Sisson et al. benchmark with emergent descriptors.	75
A.5	The 104 standardized odor descriptors (alphabetical).	76

Chapter 1

Introduction

Combining two graph-derived vectors is an unordered-pair learning problem. Each molecule can be encoded as a graph and represented by a learned vector, but a blend should produce the same prediction whether the inputs are presented as (A, B) or (B, A) . The combination method must therefore preserve input-order invariance while retaining information about the particular pair being modeled. Mean pooling is a simple order-independent baseline, but it can map different pairs to the same representation: the pairs $(2, 8)$ and $(5, 5)$ have the same average even though their elements differ. More expressive aggregation may preserve distinctions that averaging loses, but it also adds parameters and computation. Whether that added complexity produces a useful improvement is an empirical question, not an assumption of this study [44].

Fragrance-blend prediction provides the application setting for this pairwise learning problem. Graph Neural Networks (GNNs) have been used to learn molecular representations directly from structure, including in single-molecule odor prediction [24, 31]. Extending a single-molecule encoder to a two-molecule input requires a separate aggregation design. This study compares mean pooling, order-averaged concatenation, and Set Transformer aggregation for predicting catalog-recorded descriptors of binary fragrance blends [25, 41].

The blend descriptors used for training and evaluation are catalog records, not direct measurements of human perception. Public blend datasets such as the one compiled by Sisson et al. [41] are based on reference-catalog recommendations rather than a controlled sensory panel. The study therefore measures label consistency using reciprocal and cross-catalog agreement before comparing

the aggregation methods, and reports performance under declared reliability conditions rather than treating every catalog label as unquestioned ground truth.

1.1 Project Context

The fragrance domain supplies a useful application for studying this representation problem. Existing molecular GNNs can encode individual structures, but a pairwise blend task requires a second design decision: how to combine two molecule embeddings into one order-independent representation before multi-label classification [24, 41]. The unresolved issue addressed here is therefore not computational olfaction in general. It is the controlled comparison of alternative pair-combination methods under the same encoder, training conditions, and evaluation protocol.

The data structure also creates a partitioning problem. A blend dataset is organized as pair records, and the same molecule can occur in many different pairs. A random split of pair records can therefore place one molecule in both training and test partitions, allowing the model to reuse a component it has already seen. This study uses a component-disjoint split and graph-carving procedure to reduce that overlap by excluding pairs that cross the assigned partitions. The procedure is intended to reduce component leakage; it does not guarantee that every held-out molecule belongs to an entirely unfamiliar chemical family [10].

The labels create a second evaluation problem. Each blend may have several descriptors, but only a small fraction of the possible descriptor cells are positive. A model can therefore obtain a deceptively favorable aggregate score by predicting mostly absent labels, while inconsistent catalog records can change the apparent ranking of the aggregation methods. The study addresses these risks through multi-label metrics suited to sparse outputs and through a reliability audit based on reciprocal listing and cross-catalog agreement. The audit is used to define declared reliability conditions; it does not convert catalog records into verified measurements of human perception.

Fragrance and catalog terminology are retained only to identify the input domain and target descriptors. The study does not attempt to infer receptor mechanisms, explain physical mixture chemistry, or model perfume-formulation practice. Its computer-science focus is the representation and evaluation of an unordered pair of molecular graphs.

1.2 Purpose and Description

This study contributes a controlled empirical comparison of mean pooling, order-averaged concatenation, and Set Transformer aggregation under a shared molecular-graph encoding framework. It does not propose a new neural architecture. Instead, it evaluates whether an existing attention-based set aggregator provides a measurable benefit over simpler alternatives for predicting catalog-recorded descriptors of two-molecule blends [25, 41].

In all three variants, a Graph Attention Network (GAT) encodes each molecular graph into a learned vector using the same architectural specification and initialization policy [52]. The experimental variants differ in how the two molecule vectors are combined. Mean pooling averages the vectors directly. Order-averaged concatenation evaluates both input orders and averages their outputs. Set Transformer aggregation uses Self-Attention Blocks (SAB) and Pooling by Multihead Attention (PMA) to learn statistical relationships between the two vectors before producing an order-independent blend representation [25, 54].

A shared multi-label classification design maps each combined representation to independent sigmoid outputs for the target catalog descriptors. The variants are compared using matched data partitions, training conditions, and evaluation metrics. Reliability-aware evaluation reduces the influence of uncertain catalog annotations, while component-disjoint evaluation reduces the risk that the same molecular component appears in both training and test pairs. These controls strengthen the comparison, but they do not establish label correctness, eliminate all structural similarity, or guarantee that the Set Transformer will outperform the simpler methods.

Significance of the Study

The study is primarily intended to contribute evidence for computer-science research on pairwise representation learning and model evaluation. Its findings may be useful to the following groups:

- **Machine-Learning and Graph-Representation Researchers.** The study provides a controlled comparison of three methods for combining an unordered pair of graph embeddings. It contributes evidence on whether an attention-based set aggregator offers a measurable benefit over simpler methods, how model comparisons change under uncertain labels, and how component-disjoint splitting can reduce the influence of repeated molecular components across data partitions. Computational olfaction serves as the test domain for this broader represen-

tation and evaluation problem.

- **Researchers and Maintainers of Olfactory Reference Catalogs.** The reciprocal and cross-catalog agreement analyses provide a way to describe the consistency of catalog-derived labels before they are used for machine learning. These analyses may help identify descriptors that require closer review, although they do not determine whether a catalog label is perceptually correct.
- **Fragrance Researchers and Practitioners.** The results may provide preliminary evidence about the strengths and limits of structure-based prediction for catalog-recorded blend descriptors. However, this study does not validate a formulation tool or measure reductions in laboratory work, material use, or trial-and-error blending [35].

Research Questions

This study seeks to answer the following questions:

- RQ1.** Under matched encoding, training, and evaluation conditions, how does the performance of Set Transformer aggregation compare with mean pooling and order-averaged concatenation on the reliability-screened test data?
- RQ2.** How does the performance of Set Transformer aggregation relative to the two baselines vary across additive, catalog-label loss, and catalog-label gain patterns?
- RQ3.** How does the robustness of Set Transformer aggregation compare with the two baselines as training-label reliability decreases, including under controlled label corruption?

The study also has a limited connection to the United Nations Sustainable Development Goals. Its work on computational methods and data-quality evaluation is most closely related to SDG 9. Structure-based prediction could eventually support tools related to responsible material use under SDG 12, but the present study does not measure material savings, reduced laboratory work, or reduced trial-and-error formulation. These are possible future applications, not outcomes evaluated in this research.

1.3 Objectives

General Objective

To compare mean pooling, order-averaged concatenation, and Set Transformer aggregation as methods for combining an unordered pair of molecular graph embeddings for multi-label prediction of catalog-recorded blend descriptors, using matched training conditions and a reliability-aware, component-disjoint evaluation protocol.

To achieve the general objective, the following specific objectives must be accomplished:

- To prepare the single-molecule and blend datasets and describe the consistency and sparsity of their catalog-recorded labels.
- To construct fixed component-disjoint data partitions and reliability-screened evaluation conditions for comparing the models.
- To implement mean pooling, order-averaged concatenation, and Set Transformer aggregation using the same GAT encoder design, output task, loss function, and matched training protocol.
- To compare the three aggregation methods using macro F1 as the primary metric and micro F1, PR-AUC, and Hamming loss as supporting metrics.
- To determine how the relative performance of the three methods changes across label-reliability conditions, controlled label corruption, and additive, catalog-label loss, and catalog-label gain patterns.

1.4 Scope and Limitations

This study is limited to pairwise, two-molecule blends and the prediction of catalog-recorded descriptors. Each molecule is represented as a two-dimensional molecular graph and encoded by a shared Graph Attention Network (GAT) [52]. The study compares mean pooling, order-averaged concatenation, and Set Transformer aggregation as three methods for combining the resulting molecule embeddings. The Set Transformer uses attention to learn statistical relationships between the embeddings; it is not treated as a model of receptor activity or physical molecular interaction [25, 54].

The data are obtained from the GoodScents and Leffingwell single-molecule catalogs and the catalog-derived binary-blend dataset of Sisson et al. [5, 41]. The target descriptors are binary catalog records rather than intensity ratings or measurements from a newly conducted sensory panel. The study does not recruit human assessors and does not claim that its predictions represent what every person would perceive. Concentration, mixing ratio, and changes over time are also outside the scope; each blend is treated as an unweighted pair.

Multi-label learning requires both positive and negative targets. A descriptor recorded for a blend is treated as positive, while an unlisted descriptor is treated as negative under a closed-world assumption. This is an operational definition for model training and evaluation. Catalog omission may reflect an unrecorded observation rather than true sensory absence, so an unlisted cell is not treated as proof that the descriptor cannot be perceived.

Label reliability is assessed using reciprocal listing and cross-catalog agreement. These checks measure the consistency of the available records, not their perceptual truth. Reliability-screened training and evaluation use a cell-level eligibility mask: reliable recorded positives and eligible unlisted negatives are included, while ambiguous cells are excluded. This procedure can reduce the influence of uncertain annotations, but it cannot remove every form of catalog bias or labeling error.

The models are evaluated on fixed component-disjoint partitions produced by the clustering and graph-carving procedure described in Chapter 4. Pairs that cross partition boundaries are excluded so that a retained training pair and a retained test pair do not share the same molecule. This design reduces component leakage, but it does not guarantee that the partitions have no structural similarity or that the test molecules represent every possible unseen chemical family [10].

The comparison is limited to the declared model variants, metrics, reliability conditions, and controlled label-corruption experiments. The results therefore show how these methods behave on the selected catalog-derived task and should not be generalized directly to larger mixtures, other descriptor vocabularies, or human sensory judgments. Order-averaged concatenation is an adapted baseline based on the pairwise design of Sisson et al.; it is not a full reproduction of that study’s model. Final dataset sizes, target-vocabulary size, and label density will be reported only after the revised data pipeline has been run.

1.5 Definition of Terms

Molecular Graph

A computational representation of a molecule in which atoms are nodes, bonds are edges, and both carry features used by the graph encoder.

Graph Embedding

A fixed-length numerical vector produced by the graph encoder to summarize the learned information from one molecular graph.

Multi-Label Classification

A prediction task in which one blend can be assigned several descriptors at the same time. Each target descriptor is predicted separately as present or absent.

Mean Pooling

A combination method that takes the element-wise average of the two molecule embeddings. It is naturally independent of input order, but different pairs can produce the same average.

Order-Averaged Concatenation

A combination method that places the two molecule embeddings side by side, evaluates both input orders, and averages the two model outputs so that the final prediction does not depend on which molecule is listed first.

Set Transformer Aggregation

An attention-based method for combining an unordered collection of embeddings. In this study, it learns statistical relationships between the two molecule embeddings before pooling them into one blend representation.

Permutation or Order Invariance

The requirement that exchanging the positions of the two input molecules must not change the model’s final prediction.

Component-Disjoint Split

A data partition in which the same molecule does not occur in both a retained training pair and a retained test pair. It is designed to reduce component leakage, but it does not guarantee that the

partitions contain no structurally similar molecules.

Reliability Mask

A cell-level indicator that determines which blend-descriptor targets are included in reliability-screened training or evaluation. Reliable recorded positives and eligible unlisted negatives are included, while ambiguous cells are excluded.

Evaluation Metrics

Macro F1 gives each retained descriptor equal weight when averaging its F1 score. Micro F1 combines all evaluated decisions before computing one F1 score. Precision-Recall Area Under the Curve (PR-AUC) measures how well the model ranks positive targets across decision thresholds. Hamming loss reports the fraction of evaluated present-or-absent decisions that are incorrect.

Chapter 2

Methodology

This chapter describes the preparation of the catalog-derived datasets, the construction of reliability masks, the formation of component-disjoint data partitions, and the matched training and evaluation of three aggregation methods. It also defines the robustness conditions, metrics, and paired statistical comparisons used to answer the research questions. Because this is a proposal, the procedures are written in the future tense. Settings that have not yet been approved, and numbers that can only be known after the final pipeline is run, are shown as symbols or marked as pending; none of them are results.

2.1 Research Design

This study will use a quantitative experimental design on a multi-label molecular graph classification task. It compares three ways of combining the learned embeddings of the two molecules in a blend: mean pooling, order-averaged concatenation, and Set Transformer aggregation. Fragrance-blend prediction is the application. The computer-science investigation concerns how an unordered pair of graph embeddings should be aggregated, how uncertain labels should be handled, and how models should be evaluated when the two molecules of a pair may also appear in other pairs.

The research questions are the following. **[CONFIRM: copy these word for word from Chapter 1 once both authors approve them.]**

RQ1. Under matched encoding, training, and evaluation conditions, how do mean pooling, order-

averaged concatenation, and Set Transformer aggregation compare in predicting the most reliable catalog-recorded descriptors of two-molecule blends?

RQ2. How does the relative performance of the three aggregation methods vary across catalog-label pattern groups, including additive labels, catalog-label loss, and catalog-label gain?

RQ3. How does the relative performance of the three aggregation methods change when training-label reliability is varied, including under controlled label corruption?

Table 4.1 states what is varied and what is held fixed. The aggregation method is the primary varied factor. Label-reliability and label-corruption conditions are robustness factors, and the catalog-label pattern of a blend is used only for a secondary subgroup analysis. Everything else is a matched control, so that a difference between aggregation methods is not caused by a difference in data, training budget, or luck of the seed.

Table 2.1: Varied factors and matched controls.

Category	Treatment
Aggregation method	Primary varied factor (three levels)
Label-reliability and corruption conditions	Robustness factors
Catalog-label pattern	Secondary subgroup analysis
Encoder design and initialization policy	Matched control
Train/validation/test partitions	Fixed across variants
Loss, optimizer, stopping rule, training budget	Matched controls
Random seeds and corrupted datasets	Paired across variants

The dependent variables are macro F1 (the primary metric, declared before any experiment is run), micro F1, PR-AUC, and Hamming loss, defined in Section 4.7.

2.2 Data Acquisition and Preparation

This section describes the data sources, how they are joined, and how the model inputs (molecular structures) and the model targets (odor descriptor labels) are prepared. The two are prepared separately: structures become graphs, and descriptors become a fixed list of labels.

2.2.1 Data Sources and Provenance

The study will use three sources. The single-molecule sources are GoodScents and Leffingwell, both accessed through Pyrfume. They are independently maintained reference catalogs for the perfumery and flavor industries. Each record lists a molecule, its structure, and the scent descriptors assigned to it (for example, fruity or sweet). The Leffingwell descriptors are described as assigned by olfactory experts, usually practicing perfumers [24]. The blend source is the two-molecule dataset of Sisson et al. [41], obtained from their public repository.

Each Pyrfume source has three files: `behavior.csv`, which holds stimulus–descriptor pairs; `stimuli.csv`, which holds the stimulus identifier, the GoodScents or Leffingwell identifier, the PubChem CID, concentration, and solvent; and `molecules.csv`, which holds the CID, molecular weight, isomeric SMILES, IUPAC name, and name. Descriptors in `behavior.csv` are indexed only by stimulus, so they will first be joined to `stimuli.csv` on the stimulus field to obtain a CID, and then to `molecules.csv` on the CID. Concentration and solvent columns will be dropped, because concentration is outside the scope of this study.

Structure filtering. Before molecules are matched across sources, structures will be sanitized with RDKit: salts are removed, molecules containing elements other than C, H, O, N, S, F, Cl, Br, and I are excluded, and molecules outside a molecular-weight range are excluded. **[PENDING: confirm the element list and the molecular-weight range (20–600 g/mol in the earlier pipeline) and the resulting molecule count, which is a result of the final run and is not stated here.]** This step uses structure only; no label is consulted.

Join key. The two single-molecule catalogs will be merged into one master dictionary using the isomeric SMILES string as the key. Every SMILES string in both sources, and in the blend dataset, will be canonicalized with RDKit so that the same molecule written in two different ways is matched correctly. SMILES was chosen over the PubChem CID because some CIDs in the source files are negative or non-standard and would need further standardization, whereas SMILES must be canonicalized anyway when molecules are converted to graphs. Blend components in the Sisson et al. dataset, which are identified by CID, will be linked to the master dictionary by canonicalizing their SMILES in the same way.

Blend records. The blend dataset of Sisson et al. was built by crawling the “Potential Blenders” section of each GoodScents molecule page, which lists the molecules that molecule could be paired with under scent headings. The labels of a pair are the headings under which the partner was listed. **[CONFIRM: re-read the Sisson et al. Methods and supplement and state exactly how a heading becomes a blend label before this sentence is final.]** Because each molecule’s page was crawled separately, a pair can be listed from both molecules’ pages. The pair-building code of Sisson et al. counts how many pages listed a pair under each heading (one or two) but does not keep the count. This study will keep that count, because it provides the reciprocal evidence used in Section 4.3.

2.2.2 Ground Truth and Its Limits

Catalog-recorded descriptors serve as proxy targets for supervised learning. Reliability procedures estimate the consistency of those records; they do not establish perceptual correctness. The catalogs are curated reference records, not live sensory panel measurements, and their descriptors come from predefined word lists without a reported procedure for combining individual judgments. They therefore contain some noise and subjectivity [?]. This study treats them as a practical proxy for perceived scent, estimates how consistent they are, and examines how model comparisons change as label reliability changes.

For the computational scope of this study, each blend is treated as a simple physical mixture of two aroma chemicals. No significant intermolecular reaction is expected for most pairs under the assumed blending conditions (room temperature, no catalyst), so the data are treated as standard fragrance blending rather than reaction chemistry. A few highly reactive pairs may deviate from this expectation. **[CONFIRM: wording comes from chemistry faculty; permission to name the source is pending.]**

2.2.3 Target Vocabulary

Two vocabularies must be kept apart. The *standardized vocabulary* is the list of descriptor names after spelling variants have been merged. The *final target vocabulary*, of size L , is the subset of the standardized vocabulary on which models are actually trained and scored.

Standardization. The raw GoodScents and Leffingwell labels contain spelling variants of the same descriptor (for example, anisic and anise, wine and winey, or caramel and caramellic). Reviewing every label by hand would be impractical, so the study will adopt the open-source canonicalization dictionary of Sisson et al. [41], which merges raw blend descriptors into 104 standardized descriptors. **[CONFIRM: the source text gives both “approximately 400 raw descriptors” and “109 raw notes” for the input to this step; check which is correct and use one.]** Any descriptor in the merged data that the dictionary does not cover will be reviewed manually and either mapped to an existing descriptor or declared new, followed by whitespace and punctuation normalization. The standardized single-molecule vocabulary may exceed 104 descriptors if new descriptors survive this review; Appendix B will list it.

Final target vocabulary. The final target vocabulary will be built from *training data only* (Section 4.5.4). Validation and test labels will not be consulted when deciding which descriptors are kept. The size L is therefore not known yet and is written as a symbol throughout. The figure 104 is the size of the standardized vocabulary before partitioning, and the figure 74 is the number Sisson et al. reported after their own partitioning; neither is assumed to be this study’s L .

Inputs and targets. Input preparation (canonical SMILES to molecular graph with atom and bond features, Section 4.4.1) uses only molecular structure. Target preparation (descriptor standardization, mask construction, vocabulary selection) uses only descriptor labels. Neither step uses the test partition to make a decision.

2.2.4 Label Families (Reporting Only)

Expert-curated odor taxonomies, such as that of Sajan et al. [38], group raw odor words into broader families (for example, rose, jasmine, and violet under floral). The model will still predict each descriptor individually, because catalog-label loss and gain refer to specific words rather than to families. The Sajan et al. grouping is used for organized reporting only: Appendix B will map each descriptor to a family and give the number of descriptors per family. No per-family scores will be computed.

2.3 Label Reliability Mask

Not every recorded label is equally trustworthy, and not every missing label is a true absence. Rather than assigning one reliability grade to a whole blend, the study will decide, for each *blend-descriptor cell*, whether that cell is eligible for a stricter, “Gold-only” treatment. A cell is one entry of the blend-by-descriptor label matrix: one blend and one descriptor.

2.3.1 Consistency Evidence

Two kinds of evidence are used. Both measure consistency, not correctness.

- **Reciprocal-positive evidence (cell level).** A recorded positive is *reciprocal* if the pair was listed under that descriptor from the pages of both molecules (a count of two). Using the pair-building code of Sisson et al. [41], a preliminary audit found that 61,838 of 235,040 listings (26.3%) were reciprocal. **[CONFIRM: this is a preliminary audit figure; recompute with the final pipeline before reporting.]** A one-sided listing is not necessarily wrong, since catalogs do not list every possible blend, but it is not independently confirmed.
- **Descriptor-level consistency (descriptor level).** Agreement between GoodScents and Leffingwell on the molecules they share will be measured separately for each descriptor using Cohen’s kappa. In a preliminary audit on 2,225 shared molecules, 64 of 104 descriptors reached a kappa of at least 0.21 (“fair” on the scale of Landis and Koch). **[PENDING: the statistic and the cutoff are not final. The team must approve the statistic, the cutoff, and the molecule set used to compute it.]** A descriptor that meets the cutoff is called *consistent*. Reaching the cutoff does not prove that the labels are correct.

2.3.2 Eligibility Mask

Table 4.2 defines the eligibility mask $M_{ij} \in \{0, 1\}$ for blend i and descriptor j in the Gold-only treatment.

Because eligible unlisted descriptors enter as negatives, a Gold-only test set is not positive-only: a model that predicts a descriptor that is not recorded for a blend is still penalized as a false positive.

Closed-world assumption. Treating an unlisted descriptor as a negative is an operational assumption. It is needed because supervised multi-label learning requires a definition of “absent.”

Table 2.2: Eligibility of a blend–descriptor cell in the Gold-only treatment.

Cell evidence	Descriptor is consistent	Gold-only treatment
Reciprocal recorded positive	Yes	Include as positive ($M_{ij} = 1$)
One-sided or otherwise ambiguous recorded positive	Either	Mask ($M_{ij} = 0$)
Unlisted descriptor	Yes	Include as negative ($M_{ij} = 1$), under the closed-world assumption
Any cell	No	Mask ($M_{ij} = 0$)

Catalog omission may not mean true sensory absence, since a catalog may simply not list a blend or a descriptor. The reliability mask reduces this uncertainty but does not remove it, and results will be interpreted with this limitation in mind.

2.3.3 Experimental Label Conditions

Two label conditions are used.

- **Full-label condition.** Every cell is used ($M_{ij} = 1$ for all i, j), with listed descriptors as positives and unlisted descriptors as negatives.
- **Gold-only condition.** Only cells with $M_{ij} = 1$ under Table 4.2 are used.

Masked loss. In the Gold-only condition a blend is kept as long as it has at least one eligible cell; it is not removed merely because some of its cells are ambiguous. Loss is computed only over eligible cells:

$$\mathcal{L}_{\text{masked}} = \frac{\sum_i \sum_j M_{ij} \ell_{ij}}{\sum_i \sum_j M_{ij}} \quad (2.1)$$

where ℓ_{ij} is the per-cell loss (Section 4.4.3). In the full-label condition $M_{ij} = 1$ everywhere and the expression reduces to the ordinary mean loss. The same mask is applied when Gold-only metrics are computed (Section 4.7). The training, validation, and test partitions each carry their own mask, built from the same rule.

Insufficient support. After masking, a descriptor may have no eligible positives, or no eligible negatives, in a partition. F1 and PR-AUC are undefined for such a descriptor. Any descriptor with fewer than s eligible positives or fewer than s eligible negatives in the evaluated partition will

be excluded from macro-averaged scores, and the excluded descriptors will be listed in the report. **[PENDING: the minimum support s must be fixed before final evaluation.]** A training run that has no eligible cells at all is reported as a failed run and is not silently dropped.

2.4 Model Architecture

All three models share one molecular graph encoder and one output head. They differ in the step that combines the two molecule embeddings. The data flow is: each molecule is turned into a graph and encoded into a vector; the two vectors are combined by the aggregation method; and the combined vector is converted into one probability per descriptor.

2.4.1 Shared Graph Encoder

Each molecule is represented as a graph in which atoms are nodes and bonds are edges. A Graph Attention Network (GAT) encoder turns each graph into a fixed-length vector $\mathbf{z} \in \mathbb{R}^d$, so a blend yields two embeddings $\mathbf{z}_A$ and $\mathbf{z}_B$. The same encoder, with the same weights, is applied to both molecules.

[CONFIRM: node feature size, edge feature size, number of GAT layers, attention heads, readout, and embedding size d must be copied from Chapter 3 after the revised architecture is approved. The values used in the earlier notebook (46 node features, 10 edge features, 3 layers, $d = 256$) are not inherited automatically.] The attention computation is defined in Chapter 3 **[TODO: replace with \ref to the Chapter 3 section label]**. In the rest of this chapter d stands for the embedding size and, where a concrete number is needed for illustration, $d = 256$ is used.

Pretraining. The encoder may first be pretrained on single-molecule data, using a temporary head with one sigmoid output per standardized single-molecule descriptor and binary cross-entropy loss. Only molecules assigned to the training partition (Section 4.5) will be used, so molecules reserved for validation and testing are never seen during pretraining. After pretraining the temporary head is discarded and only the encoder weights are kept.

Encoder policy. **[PENDING: the team must choose one policy for the final comparison.]** The earlier notebook’s freeze/unfreeze schedule is implementation experience only and is not

the revised protocol. The two options are:

- a *frozen shared encoder*, in which encoder weights are fixed after pretraining and only the aggregation module and output head are trained; or
- the *same pretrained initialization followed by matched end-to-end fine-tuning* for all variants.

If the second option is chosen, the text will state: “The variants begin from the same encoder initialization and use matched schedules and optimization settings. Their encoder weights may diverge during end-to-end training.” In that case this study will not claim that the final models differ only in how they combine embeddings, because their encoders can end up different. Either way, the initialization policy, schedule, and training budget will be identical across the three aggregation methods.

2.4.2 Aggregation Variants

Table 4.3 summarizes the three variants. For two molecules, the embeddings form an unordered pair $\{\mathbf{z}_A, \mathbf{z}_B\}$, and a blend prediction should not change if the two molecules are swapped (permutation invariance).

Table 2.3: The three aggregation variants.

Variant	General operation	Order handling
Mean pooling	Average the two molecule embeddings	Invariant by construction
Order-averaged concatenation	Run both ordered concatenations through the head and average the outputs	Explicit symmetrization
Set Transformer aggregation	Apply set attention, then invariant pooling	Invariant set processing

Mean pooling. The blend embedding is the unweighted average of the two molecule embeddings,

$$\mathbf{z}_{\text{blend}} = \frac{1}{2}(\mathbf{z}_A + \mathbf{z}_B) \in \mathbb{R}^d. \quad (2.2)$$

This is simple and invariant to order. Sum pooling is not included as a separate variant: for exactly two molecules the sum is twice the mean, so it adds no information once the output head is trainable.

Order-averaged concatenation. The two embeddings are placed side by side, $[\mathbf{z}_A | \mathbf{z}_B] \in \mathbb{R}^{2d}$ (512 dimensions when $d = 256$). Raw concatenation depends on the order of its inputs, so the model is evaluated on both (A, B) and (B, A) and the two outputs are averaged. This follows the pairwise design of Sisson et al. [41], but it is an *adapted baseline*, not a reproduction of their whole model: it uses this study’s encoder, loss, and partitions. **[PENDING: state whether the averaging is applied to logits or to probabilities, and use the same choice in every chapter and slide. Averaging logits and applying the sigmoid once is the simpler option, but the team must decide.]**

Set Transformer aggregation. The pair $\{\mathbf{z}_A, \mathbf{z}_B\}$ is treated as a set and passed through a Set Attention Block (SAB), in which each embedding attends to both embeddings in the set (itself included):

$$\text{SAB}(X) = \text{MultiHead}(X, X, X), \quad (2.3)$$

where queries, keys, and values all come from the same set X . This lets the representation of each molecule be adjusted using the other before the two are combined. The attention mechanism follows the definition in Chapter 3 **[TODO: \ref]**. The adjusted pair is then reduced to a single blend embedding $\mathbf{z}_{\text{blend}} \in \mathbb{R}^d$ by Pooling by Multihead Attention (PMA), which uses learned attention weights instead of a plain average. **[PENDING: number of SAB blocks, number of attention heads, hidden size, and whether PMA uses one seed vector; values from the earlier notebook (2 blocks, 4 heads) are not inherited automatically.]** Both SAB and PMA are permutation-invariant in the sense needed here: swapping $\mathbf{z}_A$ and $\mathbf{z}_B$ does not change $\mathbf{z}_{\text{blend}}$. This module is a learned way of mixing two embeddings. This study does not claim that it models physical or receptor-level interaction between the molecules.

Complexity. Once the models are implemented, the study will report the number of trainable parameters of each full model and a measure of computation cost (for example, training time per epoch and inference time per batch on the same hardware), so that any accuracy difference can be read together with the cost of obtaining it. **[FILL IN: parameter counts and timings.]**

2.4.3 Output Head and Loss

The blend embedding is passed through a Multi-Layer Perceptron that maps $d \rightarrow h \rightarrow L$, where h is a hidden width and L is the final target vocabulary size (Section 4.2.3). **[PENDING: h and L ; the earlier notebook used $h = 128$.]** For order-averaged concatenation the first layer takes $2d$ inputs. The output layer applies an independent sigmoid to each of the L nodes, because odor descriptors are multi-label: each descriptor is its own present/absent decision, unlike softmax, which forces the outputs to sum to one. The framework for multi-label learning and the Asymmetric Focal Loss are defined in Chapter 3 **[TODO: \ref]**. Following Hatami and Moradi [16], the loss uses a focusing parameter of $\gamma_p = 0$ for positives and $\gamma_n = 2$ for negatives **[PENDING: confirm these values for the revised study]**. The per-cell loss ℓ_{ij} of this function enters the masked loss of Section 4.3. All three variants use the same loss, optimizer, learning-rate schedule, early-stopping rule, and training budget.

2.5 Component-Disjoint Partitioning and Graph Carving

This section describes how the data are divided into training, validation, and test sets. The aim is that no molecule that appears in a training blend also appears in a test blend. This section describes the procedure; it does not claim that the test set contains only chemically unfamiliar molecules.

Why random pair splitting leaks components. A blend dataset is a collection of pairs, and the same molecule appears in many pairs. If pairs are split at random, a molecule seen in a training pair is very likely to appear again in a test pair, so test performance partly measures memorization of molecules rather than prediction for new ones. Group-aware splitting is the standard remedy for this kind of overlap [21] (see Section 2.13).

2.5.1 Scaffold Split Decay

Scaffold splitting, which groups molecules by shared core framework, is often used for this purpose. In sparse chemical spaces, however, most scaffold groups contain very few molecules, and scaffold splitting then behaves much like random splitting (Section 2.13). To check this on the present data,

the molecule-to-scaffold ratio will be computed:

$$R = \frac{N_{\text{molecules}}}{N_{\text{scaffolds}}} \quad (2.4)$$

where $N_{\text{molecules}}$ is the number of unique canonical molecules and $N_{\text{scaffolds}}$ is the number of distinct Bemis–Murcko frameworks produced by RDKit. A value of R near 1 means almost every scaffold holds a single molecule. R will be computed on the merged master dictionary and compared with the collapse threshold $R < 2.0$ reported by Fooladi et al. [10].

2.5.2 Cluster-Based Partitions

Molecules will be grouped by structural similarity and whole groups will be assigned to one partition. The planned pipeline is: compute a binary structural fingerprint for every unique molecule; reduce it to a low-dimensional map with UMAP; and cluster the map with K-Means into $k = 7$ clusters. The number of clusters is fixed at seven, following Guo et al. [13], who found seven UMAP clusters to outperform alternative clustering methods on molecular data. A candidate fingerprint is ECFP4 (2048 bits, radius 2).

[PENDING: fingerprint type and size, UMAP settings, random seeds, and the cluster-to-partition rule are not final. They will be fixed and written down before any final evaluation, and the values of the earlier notebook will be reused only if the team deliberately re-approves them. The value $k = 7$ is the only clustering setting treated as fixed.]

The rule that assigns clusters to training, validation, and test partitions will be *declared in advance and will not depend on any label*. For example, it may depend only on cluster sizes (such as assigning clusters to partitions so that the molecule counts approximate a target ratio). **[PENDING: state the exact rule.]** The selection will not look at which descriptors occur in which cluster, and it will not look at test labels. Once the partitions are fixed they are not changed; the number of molecules, pairs, and descriptor support in each partition are reported afterwards as outcomes, not used as selection criteria. If any inspection of test labels turns out to be unavoidable, it will be disclosed, and the test set will not be described as untouched.

2.5.3 The Graph-Carving Protocol

The blend dataset is treated as a graph in which each node is a unique canonical molecule and each edge is a documented blend between two molecules. Every molecule is assigned to a partition by its cluster. Carving then examines each edge: an edge (blend) is *retained only if both molecules belong to the same assigned partition*, and discarded otherwise. After carving, no molecule is shared between any two partitions, so no molecule component can leak from training into validation or test. Sisson et al. [41] applied a similar carving to a random 50:50 molecule partition, which discarded about 83,000 pairs and left about 44,000 training and 40,000 testing pairs. This study uses clusters instead of a random molecule split, so its pair counts will differ and are reported only after the procedure is run. **[FILL IN: pairs retained and discarded per partition.]**

The validation set is used for early stopping, hyperparameter decisions, and selection of decision thresholds. The test set is kept sealed and used only in the final evaluation.

Component-disjoint partitioning reduces component and structural overlap between partitions. It does not guarantee that test molecules belong to unfamiliar chemical families, because clusters that are near each other on the map can still contain similar molecules.

2.5.4 Training-Only Vocabulary Filter

Carving changes how often each descriptor occurs, so the final target vocabulary is chosen after partitioning, from training data only. A standardized descriptor d is kept if it has enough positive examples to learn from:

$$d \in \mathcal{D}_{\text{target}} \iff c_{\text{train}}(d) \geq x \quad (2.5)$$

where $c_{\text{train}}(d)$ is the number of training blends in which d is recorded as positive, and x is a minimum-support threshold. **[PENDING: the value of x ; the earlier notebook used $x = 5$ on both training and test sides, and the test-side condition is removed here.]** The size of $\mathcal{D}_{\text{target}}$ is L . In the Gold-only condition, $c_{\text{train}}(d)$ counts eligible positives. After the vocabulary is locked, the number of validation and test positives and negatives for each descriptor is *reported* but is not used to add or remove descriptors. Descriptors that end up with too little test support are handled by the insufficient-support rule of Section 4.3 and are listed in the report.

2.6 Experiments

All experiments hold the partitions, the encoder initialization policy, the loss, the optimizer, the stopping rule, the training budget, and the metric suite constant across the three aggregation methods. They will show whether attention-based aggregation performs better than simpler alternatives; a finding that a simpler baseline performs as well or better is a valid and informative result and is reported as such. Each configuration is run with the same set of random seeds for every aggregation method. **[PENDING: the number of seeds.]**

2.6.1 Experiments A and B: Main Comparison (RQ1)

Experiment A trains each model in the full-label condition, using all cells. Experiment B trains each model in the Gold-only condition, using masked loss over eligible cells (Section 4.3). Both are evaluated on the held-out test partition. Experiment A is also scored on all cells, and Experiment B is scored on eligible cells with the same mask. Reporting both makes visible how much of any difference depends on which cells are counted.

2.6.2 Experiment C: Controlled Label Corruption (RQ3)

To examine how the comparison changes as training labels become less reliable, labels will be corrupted on purpose, in the training partition only. Validation and test labels are never changed.

For a noise level $p \in \{0, 0.10, 0.20, 0.30\}$, a fraction p of the eligible positive cells in the training labels is flipped to absent, and an equal number of eligible negative cells is flipped to present, which keeps the density of positives unchanged. Only eligible cells can be flipped, so the same mask logic as Section 4.3 applies. For each combination of noise level and random seed, one corruption mask is generated and *reused across all three aggregation methods*; a new mask is generated for every seed and noise level. The run at $p = 0$ is the uncorrupted Gold-only run of Experiment B and is reused. Models are retrained at each level and scored on the unchanged validation and test labels. The change in macro F1 from $p = 0$ to $p = 0.30$ is then compared across methods using the paired bootstrap of Section 4.8.

Random flipping tests robustness to label errors in general. It does not reproduce every real process by which catalog errors arise.

2.6.3 Blend-Type Analysis (RQ2)

To see whether the relative performance of the methods differs by catalog-label pattern, each test blend is compared with the recorded descriptors of its two parent molecules. The terms below describe patterns in the catalog records; they are not claims about what a person would perceive.

- **Catalog-label loss:** a descriptor recorded for either parent is not recorded for the blend.
- **Catalog-label gain:** the blend has a recorded descriptor that is not recorded for either parent.
- **Additive:** the blend shows neither loss nor gain.

Descriptive statistics of the label matrix (label density, common co-occurrences, and the rate at which a blend’s labels are additive with respect to its parents) are reported for the dataset as a whole. They are used to describe the data and to support this subgroup analysis; they are not used to filter data. The sizes of these three groups are reported and the model comparison is repeated within each group. Because these groups overlap in how they are defined and the analysis is split into subgroups, it is treated as a secondary subgroup and error analysis rather than a confirmatory test. If any group contains fewer than 5% or more than 95% of the test blends, blends are instead divided by the overlap between the blend descriptors and the union of the parents’ descriptors.

2.7 Evaluation Metrics

This section defines the metrics and explains why each is needed when most label cells are negative.

2.7.1 Multi-Label Metrics

Let TP be a descriptor correctly predicted present, FP a descriptor incorrectly predicted present, FN a present descriptor that was missed, and TN a descriptor correctly predicted absent. Most cells in the label matrix are negative, so a model can look accurate by predicting “absent” every time. Precision, recall, and F_1 use only TP, FP, and FN [34], so they ignore the many easy true negatives, and a model that predicts nothing receives $F_1 = 0$.

- **Macro F1 (primary metric).** F_1 is computed for each descriptor separately and then averaged with equal weight, so rare descriptors count as much as common ones [34]. This choice

is declared before experimentation and must be the same in Chapters 1–4. [**CONFIRM: Chapter 1.**]

- **Micro F1.** TP, FP, and FN are pooled across all descriptors before F_1 is computed; it summarizes overall performance and favors common descriptors.
- **PR-AUC.** The area under the precision–recall curve, which measures how well the model ranks true descriptors above false ones across all thresholds. It is reported both macro-averaged over descriptors (primary version) and micro-averaged. [**PENDING: confirm that both versions are reported.**]
- **Hamming loss.** The fraction of evaluated cells that are predicted incorrectly (below).

Thresholds. Each descriptor’s probability is turned into a present/absent prediction with a decision threshold. Thresholds are chosen on validation data only and then applied unchanged to the test data. Threshold selection matters a good deal for imbalanced multi-label problems [48]. [**PENDING: whether one global threshold or a per-descriptor threshold is used.**]

2.7.2 Masked Metrics

All metrics are computed only over cells with $M_{ij} = 1$, using the mask of the condition being evaluated (all ones in the full-label condition). For macro metrics, descriptors with insufficient eligible support are excluded and listed (Section 4.3). With a mask, Hamming loss is

$$\text{HL}_M = \frac{\sum_{i=1}^N \sum_{j=1}^L M_{ij} \mathbf{1}(y_{ij} \neq \hat{y}_{ij})}{\sum_{i=1}^N \sum_{j=1}^L M_{ij}} \quad (2.6)$$

where N is the number of evaluated blends, L is the number of target descriptors, y_{ij} is the label, $\hat{y}_{ij} \in \{0, 1\}$ is the thresholded prediction, and $\mathbf{1}(\cdot)$ equals 1 if its argument is true and 0 otherwise. Smaller values are better, with $\text{HL}_M = 0$ for a perfect match. In the full-label condition $M_{ij} = 1$ everywhere and this is the ordinary Hamming loss.

All-negative reference. Hamming loss also rewards the many easy negatives, so it cannot be read alone. The *evaluated-cell density* is

$$D_M = \frac{\sum_i \sum_j M_{ij} y_{ij}}{\sum_i \sum_j M_{ij}} \quad (2.7)$$

the fraction of evaluated cells that are positive. A model that predicts every cell absent has $HL_M = D_M$, which serves as the all-negative reference. It is computed separately for each partition (D_{train} , D_{val} , D_{test}) and for each mask, and a value from one setting is never reused in another. **[FILL IN: densities and the all-negative reference, after the pipeline is run.]** Every model’s Hamming loss is reported next to this reference.

2.8 Statistical Comparison

Differences between models could reflect chance in which blends happen to be in the test set, or chance in training. These two sources are handled separately.

Variability across seeds. Optimization variability comes from random initialization and data order. Each model is trained with the same set of random seeds, and results are summarized across seeds (mean and spread) separately from the bootstrap below.

Bootstrap over blends. Descriptor outcomes within one blend are correlated, so individual cells are not independent. The test blends are therefore resampled as whole rows: a sampled blend keeps all of its descriptor cells together, with its mask. Naive paired t -tests are not appropriate for comparing machine-learning models, because they are sensitive to outliers and not valid when resampled test sets are used [34], and bootstrap resampling shows both whether a difference exists and how large it is [?].

From the N test blends, N rows are drawn with replacement, B times **[PENDING: the number of replicates B will be fixed before final evaluation]**. All models being compared are scored on the *same* resampled indices, giving a paired comparison. For replicate b the paired difference is

$$\Delta(b) = \text{Metric}_{\text{model 1}}(b) - \text{Metric}_{\text{model 2}}(b). \quad (2.8)$$

The direction of a difference is stated with each result: higher is better for macro and micro F1 and for PR-AUC, and lower is better for Hamming loss, so a positive Δ favors model 1 for the first three and model 2 for Hamming loss. The comparison is repeated for each metric and for each pair of aggregation methods (three pairs). **[PENDING: whether a correction for multiple comparisons is applied, or whether the primary comparison is limited to macro F1.]**

Intervals and effect size. The significance level is $\alpha = 0.05$. A 95% percentile confidence interval is taken from the 2.5th and 97.5th percentiles of the B differences, and an interval that excludes zero is read as a difference at $\alpha = 0.05$. Because statistical significance does not show whether a difference matters in practice, the effect size is reported as the mean difference with its 95% interval, in the units of each metric. For Experiment C, the bootstrap is applied to the change in macro F1 between noise levels.

2.9 Reporting Conventions

Experiments that have not been run are described in the future tense. Any pipeline demonstration run before the final protocol is fixed will be labelled preliminary and will not be reported as a result. Output from the earlier notebook is not reported as a result of the revised study. The study does not assume that the Set Transformer will outperform the baselines; if mean pooling or order-averaged concatenation matches or exceeds it, that outcome will be reported and discussed on equal footing. Quantitative Structure-Odor Relationship (QSOR) modeling refers to the prediction of the odors and olfactory properties of molecules through the characterization of their structure [12, 50, 55]. It has long been regarded as one of the key challenges in olfactory research in machine learning [5, 29, 52]. QSOR is especially important in fields such as the fragrance and flavor industries, as it could help streamline the formulation and development of new compounds computationally, as opposed to traditional experimental wet lab methods, which can be both time-consuming and expensive [17, 37].

Early approaches in QSOR involved the manual creation of numerical features that describe and quantify the properties of a molecule [3, 12, 56]. These included molecular fingerprints, which encode the presence or absence of substructures in a binary vector, and physicochemical descriptors computed with dedicated software [3, 9, 39, 49].

Many studies pair such descriptors or fingerprints with classical learners such as random forest, XGBoost, and LightGBM to predict odor [37, 43, 48]. Two comparisons are representative. Suh et al. found that Morgan-fingerprint models outperformed descriptor-based ones on a curated set of 8,681 fragrance compounds [43], and Achebouche et al. found that graph convolutional representations outperformed general molecular descriptors and fingerprints on a set of 5,955 molecules [1].

The common denominator across these approaches is that they convert the molecule, specifically

its properties, into fixed, handcrafted numerical representations, which are then fed into classical ML models. Recent work such as Suh et al., Achebouché et al., and Wen et al. continues to include descriptor-based classical models, but mainly as baselines to quantify how much fingerprint- and graph-based learning improves upon manual feature engineering [1, 43, 48]. While effective, these methods across both eras require the manual selection of descriptors in advance [4, 24, 56]. This means the model is not actually able to determine on its own which structural features are relevant for the task it is trying to solve. This necessitated the advancement to graph-based deep learning methods, which improve the odor prediction pipeline by teaching models to learn task-relevant features directly from structure [2, 4, 15, 55], which are discussed next.

2.9.1 The Principal Odor Map

Addressing the aforementioned limitation is Lee et al.’s Principal Odor Map (POM), which maps a molecule’s chemical structure to its smell using Graph Neural Network (GNN) techniques [24]. Rather than evaluating a molecule’s scent using a static representation built from predefined handcrafted descriptors, POM learns directly from the molecule’s structure during training. It does this by treating each molecule as a graph, where nodes and edges represent atoms and chemical bonds, respectively; each node and edge is itself a vector, with nodes storing properties such as atomic number, and edges storing bond properties.

Through message passing, the GNN dynamically assigns weights to each of the molecule’s structural features across multiple passes, producing a numerical representation of the molecule. The final layer predicts the molecule’s specific odor qualities, while the penultimate layer forms the actual Principal Odor Map, which the paper itself describes as a high-dimensional, data-driven representation that is able to preserve human perceptual distances and hierarchies, positioning molecules with similar smells closer together in the generated odor space [24].

The setup was as follows: Lee et al. used approximately 5,000 molecules derived from merging data from the GoodScents and Leffingwell fragrance databases to produce a multi-label output of 138 distinct semantic odor descriptors. It was then validated against a human benchmark, wherein approximately 15 trained panelists rated molecules using a narrowed-down set of 55 odor descriptors. The results show it was able to match the trained panel’s verdicts on every label except for musky scents, and was even able to surpass the median panelist on 30 out of the 55 validation labels (55%) [24].

This demonstrates that graph representations can encode predictive scent information from molecular structure alone. However, despite this significant advancement, these results were derived exclusively from single, unmixed molecules.

2.9.2 Limitations of Single-Molecule Models

POM shows what a graph encoder can learn for one molecule, but it operates on a single molecule at a time, while odorants rarely appear in isolation [7, 24, 46, 53]. Real-world scents are rarely composed of single molecules; rather, they are blends that can contain individual components numbering up to the hundreds [14, 53]. The limitation is computational: a single-graph encoder produces one vector per molecule, and it does not define how the vectors of two molecules should be combined into one prediction. That combining step is the subject of this study.

Lee et al. admit this limitation themselves, noting that while POM uses a coordinate system to categorize scents, it alone cannot determine the behavior and outcomes of different molecules when mixed [24].

2.10 Psychophysics of Odor Mixtures

Because single-molecule models do not specify how two molecules are combined, a method for combining them is needed. This section establishes only that both simple averaging and interaction-aware aggregation are plausible candidates for that method. It reviews how catalog blend labels differ from the labels of their parent molecules, and the human-panel evidence that averaging may be sufficient. It does not claim that either method reproduces perception.

2.10.1 Scent Blending Mechanics

A paper released by Sisson et al. introduces a labeled dataset consisting of molecule pairs to measure the divergence of the emerging scent when two molecules are mixed from their original scent labels, using two metrics: union and intersection. The union score operates under the assumption that the emerging scent will smell like any of the scents either parent molecule has (additive mixing). On the other hand, the intersection score operates under the assumption that the outcome of the two molecules when mixed will only come from the scents the parent molecules share (subtractive mixing) [41].

The Jaccard results show 0.12 for union and 0.24 for intersection [41]. These low scores indicate that neither additive nor subtractive mixing reflects the catalog blend labels. This suggests that simple union or intersection rules do not reproduce the catalog-recorded labels of molecule pairs.

Catalog blend labels differ from the labels of the parent molecules in two ways. In *catalog-label loss*, a scent word recorded for either parent molecule is absent from the blend’s labels. In *catalog-label gain*, the blend’s labels contain a scent word that neither parent has. The results of Sisson et al. show that, on average, 1.2 notes are lost and 0.2 notes are gained per blend. Furthermore, an odds-ratio analysis reveals that certain scents, such as “eggy” or “cabbage”, were more likely to be gained, while certain scents like “peach” or “fresh” were more likely to be lost [41]. These are differences between catalog records, not verified perceptual effects. They appear to follow a pattern rather than being random, which motivates testing a model that can learn relationships between the two molecules, such as a Set Transformer.

2.10.2 Evidence That Averaging May Be Sufficient

Not all evidence points to strongly non-additive mixing. Pellegrino et al. [32] had a trained panel rate 432 mixtures using a 51-term list and found that a linear average of the components’ ratings predicted mixture profiles about as well as the noise in the panel’s own ratings allowed. They argue that apparent emergence often reflects the limits of describing an odor with a few words. Binary catalog labels, as in Sisson et al. [41], instead record catalog-label loss and gain in many blends. Whether an interaction step helps for binary multi-label prediction is therefore an open question, which this study tests rather than assumes.

Human studies also describe two ways in which a mixture can be perceived: elementally, where its components can be picked out, and configurally, where the mixture has a quality of its own [8, 36]. Which one dominates can depend on the proportions of the components, and because this study does not model concentration, each blend is treated as an unweighted pair. Odorant antagonism and masking have been demonstrated for some odorants [11, 19, 20]. This study does not model receptors; the Set Transformer is evaluated only as a statistical aggregator of two learned vectors.

2.11 Machine Learning on Sets and Chemical Mixtures

This section is the core of the chapter. It defines permutation invariance, explains how mean pooling, concatenation, and the Set Transformer each combine two molecule vectors, and reviews prior work on mixture modeling.

2.11.1 Permutation Invariance on Sets

A combination of two or more molecules could be viewed as a mathematical set. An accurate depiction of a fragrance blend of these molecules requires that they be presented in such a way that their positioning in that list is arbitrary, not sequential [41, 46, 55]. For instance, assume the combination of molecule A and molecule B produces scent C. It should be of no relevance whether it is molecule A + B or molecule B + A; it should always produce scent C. For that reason, it is important for any mathematical model used to predict the resultant smell from the combination of different molecules to follow this principle, which is called permutation invariance. In plain terms, a function is permutation invariant if swapping its inputs does not change its output.

Zaheer et al. [54] established the math of permutation invariance and introduced Deep Sets, a deep learning framework that uses a neural network designed to process datasets in a non-sequential manner for unordered data, which serves a different function from architectures that must process data in order (e.g., molecular compounds vs. protein sequences). This is referenced as the first of two properties they introduced in their study describing permutation invariance, positing that for a function acting upon a set, it must be indifferent to the order in which each element appears, mathematically formulated as $f : 2^X \rightarrow Y$ for any permutation, i.e., for any permutation π :

$$f(\{x_1, \dots, x_M\}) = f(\{x_{\pi(1)}, \dots, x_{\pi(M)}\}).$$

Their second result explains how to build such a function: transform each element individually (ϕ), aggregate the results, and then transform the aggregate again (ρ) [54].

However, because each element is transformed before pooling and in isolation, any information about how the elements relate to each other must be inferred indirectly by later layers from the pooled representation, as noted by Lee et al. [25]. For a pair of molecules, this raises the question of whether letting the two vectors affect each other before pooling helps.

To bridge this gap, Lee et al. propose the use of a Set Transformer, where instead of processing the elements of a set individually, they apply self-attention mechanisms they call SAB (self-attention

block) in the processing of the elements, wherein the different elements are able to learn from each other, such that the extracted features to be used for pooling already contain useful information about the elements’ relationship with each other. Moreover, they also developed PMA, or Pooling by Multihead Attention, which replaces the second step in the Deep Sets architecture wherein, instead of fixed sum or mean pooling, PMA also employs self-attention mechanisms in the pooling itself, such that it is able to pick out which of the elements bear more weight for the task at hand. Both are candidates for combining the two molecule vectors in this study: SAB lets each vector be updated using the other before pooling, and PMA learns how to weight them. The Set Transformer is therefore treated as a way to model statistical relationships between two learned vectors, not as a model of masking, emergence, or receptor behavior.

Three ways of combining the two vectors are compared in this study. *Mean pooling* averages the two vectors. It is simple and naturally order independent, but it can lose information specific to the pair, because different pairs with the same average produce the same combined representation [44]; this does not mean that it must fail on any particular task. *Concatenation* places the two embeddings side by side, as in the pairwise model of Sisson et al. [41]. It keeps both embeddings in full, but its output depends on which molecule is listed first. This study therefore symmetrizes it by evaluating both orders and averaging the two outputs, which is called *order-averaged concatenation*. *Set Transformer aggregation* lets the two vectors affect each other’s learned representation before pooling, while remaining independent of order.

The three methods also differ in cost. Self-attention blocks in the Set Transformer scale quadratically with the number of elements in a set [25]; for a set of two molecules this cost is small, but attention blocks still add trainable parameters and computation compared with fixed pooling. A fair comparison therefore reports parameter counts and computation cost alongside accuracy, so that any gain can be weighed against the extra complexity.

2.11.2 Precedents in Mixture Modeling

Zaheer et al. [54] provided the groundwork for permutation invariance in sets, while Lee et al. [25] extended it with self-attention, a general set method that later studies applied to molecular mixtures. Plenty of studies have made use of their work in different realms of chemistry.

For example, Qin et al. [33] built SolvGNN to predict activity coefficients of binary and ternary chemical mixtures using two levels of graph processing: graph convolutions inside each individual

molecule, followed by a molecule-level graph whose nodes are the molecules themselves. They argue that modeling the mixture explicitly at this second level matters in cases where naive pooling falls short [33].

Similarly, Sanchez Medina et al. [39] introduce GH-GNN, which also predicts activity coefficients of chemical mixtures. It uses a single shared molecular encoder whose pooled output is passed to a mixture-level graph. This resembles the structure of the present study, a shared single-molecule encoder followed by a mixture-level step, but applied here to odorant pairs rather than to solvent and solute.

Lastly, Tom et al. [46] proposed POMMix, a more relevant precedent for our topic, as it is in the field of olfactory perception, aiming to predict the similarity of the scent of two mixtures. Theirs employs a 3-layer architecture, with the first making use of the previously discussed POM, where a GNN processes the atomic bond traits of the molecules along with the scent labels to create a multi-dimensional vector that describes the scent of the molecule. The second layer employs a module they call CHEMIX, which uses a self-attention mechanism across the graph from the first layer, doing set-pooling to produce a permutation-invariant mixture embedding that is able to tell which components contribute to the nonlinear behaviors that odorants exhibit. The last layer of their architecture is the cosine predictive head, which evaluates the similarity of mixtures according to their scent by calculating the scaled cosine distance between their respective embeddings. Their ablation study compared several molecular embedding types including handcrafted RDKit descriptors and pretrained language-model embeddings, combined with both a classical model (XGBoost) and their attention-based CHEMIX module, finding that embeddings derived from POM consistently outperformed the alternatives, suggesting that structure-aware, attention-based representations are useful for mixture modeling. POMMix differs from this study in its prediction task: it predicts the similarity between two mixtures, whereas this study predicts the scent labels of a two-molecule blend.

These three studies tackle different prediction problems. Evidence from solvent mixtures in particular does not transfer directly to scent labels. They motivate evaluating interaction-aware aggregation for odor blends; they do not establish that it outperforms simple pooling for the binary scent labels used here.

Hierarchical GNNs have also been used for property prediction within a single molecule, moving from atoms to fragments, as in HiGNN and HimGNN [15, 56]. POMMix, SolvGNN and GH-GNN,

in contrast, operate across molecules. The present study follows the second kind: a shared single-molecule encoder followed by an interaction step between the two molecules.

2.12 Olfactory Datasets and Label Reliability

This section distinguishes the single-molecule catalog labels from the pairwise blend labels, describes what each can and cannot support, and explains why label inconsistency, label noise, and label sparsity matter for evaluation.

2.12.1 Reference Catalogs

The quality of sources for public scent data are not equal. Community platforms such as *Fragrantica* and *Basenotes* describe finished commercial perfumes in the words of their users, which makes them unsuitable for learning how individual chemicals relate to scent [5, 49]. Because of these, researchers use catalogs such as *GoodScents* and *Leffingwell* for reference, as they record descriptors for individual aroma chemicals and have been used to train state-of-the-art prediction models [24, 41]. The *Leffingwell PMP 2001* database was compiled by olfactory experts, usually practicing perfumers, and its free-text descriptions were standardized into 113 descriptors by Sanchez-Lengeling et al.

2.12.2 The Blended-Pair Dataset

Sisson et al. [41] compiled blend labels from the lists of suggested blending partners on *GoodScents*, which group each partner under an odor heading. Their crawl was able to produce more than 160,000 pairs and, after standardization and filtering, resulted in up to 74 odor labels. Each molecule’s page was crawled separately, so a pair can be listed from both molecules’ pages; their pair-building code counts these listings but does not keep the count. The dataset has no concentration information, and its labels are catalog suggestions rather than measurements from a sensory panel. In this study they are therefore treated as catalog-recorded blend labels and used as proxy targets. Reciprocal listing and cross-catalog agreement are used as evidence of label consistency; they do not establish that a blend label is a correct description of human perception.

2.12.3 Human-Rated Datasets

Expert-compiled catalogs such as Leffingwell are assigned by people, but they are used here as training data and cannot serve as an independent check. Independent human-rated data are therefore needed. Human-rated scent datasets exist for single molecules and for mixtures, but to our knowledge none that gives scent descriptors for two-molecule blends is publicly available at the time of writing. Bierling et al. [?] provide yes-or-no ratings of 16 scent words for 74 single odors from about 1,200 laypeople. Pellegrino et al. [32] collected trained-panel ratings of mixtures with a 51-term list and have stated that their data will be released after publication [personal communication].

2.12.4 Absence of a Standard Vocabulary

No standard set of odor descriptors exists. Catalogs and studies use vocabularies of different sizes, and descriptors overlap in meaning [41]. To help structure this vocabulary problem, Sajan et al. [38] published an expert-curated taxonomy that groups raw odor words from the Pyrfume repository into 16 broader scent families. This kind of high-level grouping is useful for organizing descriptors across studies. This study maps its labels to these families for organized reporting only (Appendix B). The models still predict individual words, and no result or conclusion depends on the family mapping.

2.12.5 Noisy and Imbalanced Labels in Multi-Label Learning

Inconsistent labels act as noise in both the training targets and the test targets. Noise in the training targets can change what a model learns, because deep networks can memorize noisy labels [?]. Noise in the test targets can change which model appears best. Noise is harder to handle with several labels per example, because errors can be correlated across labels; Li et al. [27] model this by estimating a noise transition matrix that uses label correlations. For this reason, this study measures label consistency, evaluates on the most consistent labels, and tests whether the comparison between aggregation methods changes as label reliability decreases.

Multi-label scent data are also sparse: most descriptors are absent for most molecules, and some descriptors are far more common than others. Reviews of imbalanced multi-label classification describe imbalance within labels, among labels, and among label sets [45], and work on graph neural networks reports that the benefit of balancing techniques depends on the dataset and problem type

[2]. Because simple accuracy-style scores can look good on sparse labels, evaluation of such data needs metrics that are not dominated by true negatives [34].

2.13 Out-of-Distribution Splitting Strategies

This section will be the last technical segment of this chapter on data, providing a review of the standard data-splitting strategy of scaffold splitting, and an examination of how its application falls short in sparse chemical spaces, which then motivates the need for more rigorous out-of-distribution (OOD) evaluation. This section then grounds and discusses the clustering-based alternative our proposed framework will employ.

A common method to partition data for training, testing, and validation in chemoinformatics and molecular machine learning is scaffold splitting [13, 42]. In this method, molecules are represented by graphs of their structure, and the core framework of these graphs is referred to as their “scaffold” (Bemis–Murcko scaffolds) [51]. Molecules that share the same scaffold are grouped together, and these groups are then distributed among the training, testing, and validation sets to ensure that the test set contains structurally different molecules from the training set, which is a basic form of out-of-distribution (OOD) evaluation, which as opposed to random splitting, simply distributes molecules randomly across training, validation, and test sets [10, 13, 41, 42].

The caveat with this method is that, when the dataset is sparse, it collapses toward random splitting. For instance, similarly structured molecules get fragmented across both the train and test splits, leading to severe data leakage and inflated test scores as scaffolds in standard benchmarks could be populated by only one or two molecules each, undermining the grouping effect the method depends on [10]. Another problem demonstrated by scaffold splitting is that even molecules that are near-identical (as computed by Tanimoto similarity) [13, 42], even differing by as little as a single atom, can, under this method, end up in opposite splits, as observed by Guo et al. and Steshin et al. [13, 42], who argue that this problem causes models to “memorize” rather than generalize from the data. Because of these problems, scaffold splitting can misleadingly produce scores that are overly optimistic.

Shimakawa et al. [40] propose doing topological pre-clustering by projecting 2048-bit ECFP4 fingerprints onto a 2D map using 2D Uniform Manifold Approximation and Projection (UMAP), then grouping them together using spectral k -means clustering. This was then validated using Leave-

One-Cluster-Out Cross-Validation (LOCOCV), where clusters are left out in rotation for training and testing, such that what the model was trained on does not show up in what it is tested on, meaning the model actually learns, rather than simply memorizing the data it was trained on, which is important for OOD evaluation. More generally, leakage-reduced splitting can be formulated as an optimization problem over the data [21], and cluster-based splits have been reported to be among the most challenging for both classical and graph-based models [10].

In plain terms, one molecule can appear in many blend pairs, so splitting pairs at random can place the same molecule in both training and testing. Methods designed for single molecules do not address this, because an individual molecule can be a component of varying other molecules in different blends, meaning a molecule seen during training could reappear as a component in a different, unseen test blend, leaking information about that molecule across the split and inflating the performance metrics. Therefore, Sisson et al. [41] propose “graph carving”, using a meta-graph to describe the pairwise blend dataset, where each node represents an individual molecular graph and each edge represents a tested blend combination. Edges connecting a molecule from one partition to a molecule in another are discarded, so that no component is shared between splits. In their original implementation, this partitioning was done by randomly assigning molecules to each side and repeating until every label had at least one training and one test example, since a true scaffold-based split was not compatible with their sparse label distribution.

Building on this, our proposed framework instead applies the graph-carving protocol on top of a fingerprint-clustering partition (in the style of the UMAP and k -means approach discussed above), rather than random or scaffold-based partitioning. Clustering molecules and then carving the resulting blend meta-graph along those cluster boundaries yields a component-disjoint split. This reduces component and structural overlap between training and testing; it does not guarantee that every test molecule is entirely unlike every training molecule.

2.14 Evaluating and Comparing Models Under Label Sparsity

Sparse multi-label targets make some common scores misleading. Precision, recall, and F1 ignore true negatives and so expose a model that predicts no labels, whereas accuracy-style scores such as Hamming loss can look strong for such a model [34]. Threshold choice also matters: in a comparison

of graph and classical models for multi-label odor prediction, optimizing the decision threshold was reported to be essential [48].

Comparisons between models should also report how large a difference is, not only whether it is statistically significant. Bootstrap confidence intervals do this by showing the difference and its uncertainty together [?], and recent guidance cautions against naive paired t -tests when resampled test sets are used [34]. Together with the leakage problem above, this means that a fair comparison of aggregation methods needs matched encoders and training conditions, sparsity-aware metrics, validation-only threshold selection, and paired uncertainty estimates over the evaluated blends.

2.15 Synthesis and Research Gap

2.15.1 Synthesis of Literature

Current models in computational olfaction can successfully map single-molecule structures to their appropriate sensory descriptors [24]. However, because these compounds are evaluated in isolation, they do not address how two molecules should be combined, and catalog blend labels differ from the labels of the parent molecules in many blends [41]. Human-panel evidence on mixtures is mixed: continuous ratings are well predicted by a linear average of the components [32], while binary catalog labels record catalog-label loss and gain in many blends [41].

From an architectural standpoint, permutation-invariant set processing is needed to predict the behavior of multi-molecule mixtures [54, 25]. Simple linear pooling (mean or sum) can make different pairs look identical, because pairs with the same average produce the same combined representation [44].

Furthermore, standard scaffold splits suffer from severe data leakage at the component level in mixture spaces [10]. Clustering molecules and carving the blend meta-graph along cluster boundaries reduces component and structural overlap between training and testing [40, 41]. Finally, the blend labels in public data come from catalogs rather than sensory panels, and their reliability is rarely measured.

In summary: (1) a molecular graph encoder produces one vector per molecule; (2) the unresolved computing problem is how to combine the two vectors without making the prediction depend on input order; and (3) any such comparison must account for component leakage and uncertain labels.

2.15.2 Statement of the Research Gap

Existing work does not yet establish whether an attention-based set aggregator provides a reliable advantage over simpler alternatives when two molecular graph embeddings are combined for sparse, catalog-derived multi-label prediction. A fair answer requires the encoder and training conditions to be controlled, input order to be handled explicitly, repeated molecular components to be separated across partitions, and uncertain labels to be considered during evaluation. This study addresses that controlled evaluation gap.

2.15.3 Positioning of the Present Study

To address this gap, this study compares mean pooling, order-averaged concatenation, and Set Transformer aggregation for combining the two molecule embeddings produced by a weight-shared graph attention encoder. The comparison is carried out on a component-disjoint, fingerprint-clustered and graph-carved partition of the data; the label-reliability analysis and the controlled-noise tests show how far the conclusions depend on the labels. A simpler baseline that matches or outperforms the Set Transformer would be a valid and informative result.

Chapter 3

Theoretical and Conceptual Framework

3.1 Theoretical Framework

This section presents the theory behind the network architecture and the label reliability analysis. It first describes how each aroma molecule is represented as a graph of atoms and bonds. It then outlines the foundations of Graph Neural Networks (GNN) and the message-passing framework, followed by Graph Attention Networks (GAT), which encode each individual molecule. The subsequent part details set invariance and Set Transformers, which combine the two molecules of a blend. The fourth part explains why simple averaging can lose information about how the two molecules relate to each other, which motivates testing an interaction layer against averaging and concatenation baselines. Because this argument describes a worst case and not an observed result, whether the interaction layer improves prediction is treated as an empirical question. The final part covers the measures used to assess label reliability: reciprocal agreement, Cohen’s kappa, and learning from noisy labels.

3.1.1 Graph-Theoretic Molecular Representation

Each aroma molecules is modeled as an undirected, attributed molecular graph in which atoms are nodes and bonds are edges, and create a mathematical representation where atomic and bond

features are updated via message passing between the connected atoms [52]. This would allow the Graph Attention Network (GAT) to extract structural scent profiles regardless of atom ordering. In cheminformatics, chemical compounds are often written as one-dimensional character strings, such as text-based Simplified Molecular Input Line Entry System (SMILES) strings. However, these text-based strings are syntax-sensitive. A single molecule can be written as many as single valid string depending on the starting atoms, and small edits can produce a completely different molecule [31, 49]. Representing a molecule as a graph captures its bond connectivity without depending on the order in which atoms are listed, which matches how the Graph Attention Network passes information between bonded atoms.

Mathematical Formalism

Each aroma molecule will be defined as an undirected, attributed graph $G = (V, E)$, where:

- V represents the finite set of $N = |V|$ nodes, corresponding to the non-hydrogen atoms in the molecule. Hydrogen will be treated as implicit properties embedded directly within the feature vectors of their parent heavy atoms, rather than independent nodes, which will reduce the graph complexity and computational overhead [4].
- E represents the set of $M = |E|$ undirected edges. An edge $(v_i, v_j) \in E$ represents a covalent bond between heavy atoms v_i and v_j . Because a bond is a mutual sharing of electrons between two atoms, it has no direction: $(v_i, v_j) \in E$ if and only if $(v_j, v_i) \in E$ [4].

The connectivity of the molecule is described by a symmetric binary adjacency matrix $A \in \{0, 1\}^{N \times N}$, where $A_{ij} = 1$ if a bond exists between atoms v_i and v_j and $A_{ij} = 0$ otherwise [56]. In implementation, the graph is stored in a sparse format instead of a dense matrix (Section 3.3.2). Each node and edge also carries a feature vector:

- Node vector $\mathbf{x}_i \in \mathbb{R}^{46}$, which is the initial state $\mathbf{h}_i^{(0)}$ used by the GAT. It records the atom's element, formal charge, number of bonds, valence, hybridization, aromaticity, attached hydrogens, ring membership and ring size, and chirality, in 9, 1, 6, 7, 6, 1, 5, 6+1, and 4 dimensions respectively, for a total of 46 (Table A.1) [?, 56].
- Edge vector $\mathbf{e}_{ij} \in \mathbb{R}^{10}$, which records the bond's type, conjugation, ring membership, and stereochemical configuration (Table A.2) [56].

Olfactory Domain Mapping

Some structural details are known to affect scent, including chirality (the two enantiomers of carvone, for example, smell of spearmint and caraway), ring structure, and aromaticity. The node and edge features encode these details so that the model has access to them. Features are computed with the RDKit cheminformatics library [5, ?, 24], using feature types common in molecular and odor-prediction models [24, ?]. The features describe individual atoms and bonds. Larger structures such as functional groups are not encoded directly; the GAT can learn to recognize them by combining information across neighboring atoms. The exact composition of the 46-dimensional node feature vector and the 10-dimensional edge feature vector is given in Appendix A.

Analytical and Methodological Justification

Aroma molecules modeled as graph structures instead of physical descriptors or using the text-based SMILES strings provide key advantages:

- **Order Independence:** The encoder sees the molecule as one connected structure as a whole, so the output does not depend on the sequence in which atoms are listed. This concerns atom order within one molecule and is separate from the order independence of the two-molecule blend discussed in Section 3.1.4. This ensures that it predicts scents no matter how the atoms are arranged [31].
- **Localized Feature Preservation:** Specific clusters of atoms, called “osmophores,” are thought to be responsible for particular odor qualities [?]. Since each atom carries its own features and the GAT aggregates information from bonded neighbors, the encoder can learn patterns in local chemical environments before combining them into a molecule-level embedding [52, 24]. The model is not given receptor data, so it does not identify receptor-binding sites.
- **Isomers:** Molecules with similar atoms but different bonding—*isomers*—have different graphs and are therefore distinguished. Molecules that have the exact same atoms and bonding patterns (*stereoisomers*) are distinguished because explicit attributes—chirality and double-bond configuration—are incorporated into the feature vectors, addressing the fact that plain 2D topological graphs do not represent 3D conformation [4, 49, 55].

3.1.2 Foundations of Graph Neural Networks and Message Passing

Most neural networks expect inputs arranged in a fixed form, such as the pixels in an image. A molecule has no such fixed arrangement: it can contain different numbers of atoms, and the important relationships are the bonds between them. A Graph Neural Network (GNN) represents a molecule as a graph, where atoms are nodes and bonds are edges. This lets the model use the molecule’s structure directly instead of first replacing it with a list of hand-designed measurements [56, 4, 48].

The main operation in a GNN is message passing. At each layer, an atom receives information from the atoms connected to it, combines that information, and updates its own representation. For atom v_i at layer t , this can be written as:

$$\begin{aligned}\mathbf{m}_i^{(t)} &= \bigoplus_{j \in \mathcal{N}(i)} M_t(\mathbf{h}_i^{(t)}, \mathbf{h}_j^{(t)}, \mathbf{e}_{ij}), \\ \mathbf{h}_i^{(t+1)} &= U_t(\mathbf{h}_i^{(t)}, \mathbf{m}_i^{(t)}).\end{aligned}\tag{3.1}$$

where $\mathbf{h}_i^{(t)}$ describes atom i , $\mathbf{e}_{ij}$ describes the bond to neighboring atom j , M_t produces the information sent along that bond, $\bigoplus$ combines information from all neighboring atoms, and U_t updates the atom’s representation. The aggregation does not depend on the order in which those neighbors are listed. As a result, the same molecule receives the same representation even when its atoms are entered in a different order [56, 15].

Repeating this operation allows information to move from an atom to nearby parts of the molecule and then to larger structural groups. After the final layer, the atom representations are combined into one molecule representation using a readout step. In this study, mean pooling is used for that readout, producing the 256-dimensional vector passed to the blend-level models. Recent molecular GNNs use the same atom-to-molecule process, while hierarchical models extend it by learning representations at both atom and substructure levels [15, 48].

GNNs differ in how they decide which neighboring atoms should receive more attention. Some graph-convolution methods use fixed weights based on graph connectivity. A Graph Attention Network (GAT), used in this study, learns these weights from the atom and bond features. This gives the Level 1 encoder a way to emphasize the local structures that are useful for predicting scent labels [4, 48]. These learned weights are parameters of the model; they are not evidence that the model has identified olfactory receptors or proved a chemical mechanism. The GAT operation is described in the next subsection.

3.1.3 Deep Learning Theory: Graph Attention Networks (GAT)

The model uses a Graph Attention Network (GAT) to map each molecular graph to a numerical vector. Atoms are the nodes and the bonds connecting them are the edges. Instead of combining a neighboring atom’s information with fixed weights set by the graph structure, a GAT learns a weight for each neighbor, so the encoder can give more influence to some neighboring atoms than others when building its vector representation [24, 52]. This is loosely analogous to olfactory receptors responding more strongly to some structural features than to others, but the learned weights are model parameters and are not evidence of receptor behavior or chemical importance.

Let $\mathbf{h}_i^{(0)} = \mathbf{x}_i$ denote the initial 46-dimensional feature vector of atom v_i . A learnable weight matrix $\mathbf{W}$ projects the features of each atom into a hidden space of size $F' = 256$. To score how much atom v_i should attend to a neighbor v_j , a single-layer feedforward network with parameters $\mathbf{a}$ evaluates the concatenated projected features of the two atoms and the projected 10-dimensional bond feature vector $\mathbf{e}_{ij}$ between them [52, 56]:

$$\tilde{e}_{ij} = \text{LeakyReLU}\left(\mathbf{a}^T[\mathbf{W}\mathbf{h}_i \parallel \mathbf{W}\mathbf{h}_j \parallel \mathbf{W}_e\mathbf{e}_{ij}]\right), \quad (3.2)$$

where $\parallel$ denotes concatenation. Including $\mathbf{e}_{ij}$ lets the score depend on the bond type, conjugation, ring membership, and stereochemical configuration. The LeakyReLU, with a negative slope of 0.2, keeps a small gradient for negative scores so that they do not become stuck at zero [52].

Atoms have different numbers of neighbors, so the raw scores are normalized across the neighborhood $\mathcal{N}(i)$ of atom v_i with a softmax, so that the weights for each atom sum to one [52, 4]:

$$\alpha_{ij} = \text{Softmax}_j(\tilde{e}_{ij}) = \frac{\exp(\tilde{e}_{ij})}{\sum_{k \in \mathcal{N}(i)} \exp(\tilde{e}_{ik})}. \quad (3.3)$$

The normalized weights α_{ij} scale the projected neighbor features, and the Exponential Linear Unit (ELU) activation is applied to give the updated representation of atom v_i [52]:

$$\mathbf{h}_i^{(l+1)} = \text{ELU}\left(\sum_{j \in \mathcal{N}(i)} \alpha_{ij} \mathbf{W}\mathbf{h}_j^{(l)}\right). \quad (3.4)$$

To stabilize learning, the encoder runs $K = 4$ independent attention heads in parallel. In the hidden layers their outputs are concatenated, and in the final layer they are averaged [52]. Global mean pooling over the atoms of the molecule then gives the final 256-dimensional molecular embedding.

3.1.4 Set Invariance and Set Transformers

Mixing two fragrance ingredients regardless of their order is presumed to smell the same. Mathematically, mixing Molecule A and Molecule B will have the exact same scent as mixing Molecule B and Molecule A. The blend follows the commutative property, which means that it behaves as an unordered set where the order of input does not change the output. A blend is therefore treated as an unordered set of two molecules rather than an ordered pair, and the function that combines them is required to be permutation-invariant, so that its output does not depend on which molecule is listed first [41, 54].

The Deep Sets Formulation

Deep Sets addresses this order-independence problem. It shows that this framework guarantees permutation invariance by processing each set element through an identical neural network transformation before aggregating their representations using a symmetric function, such as summation and averaging [54]. Formally, a function $f(X)$ operating on a set of molecular embeddings X is defined as:

$$f(X) = \rho \left(\sum_{x \in X} \phi(x) \right). \quad (3.5)$$

The function ϕ represents an encoder network that projects each individual molecular embedding x to a latent representation, while the sum operator aggregates these individual representations regardless of the order they are summed in [54]. In this study, ϕ corresponds to the Level 1 GAT encoder, which produces a 256-dimensional embedding for each molecule independently; summation or averaging over the two resulting embeddings is one candidate for the aggregation step that ρ applies to, evaluated here as the averaging baseline. A limitation of this formulation is that ϕ evaluates each molecule in isolation: any interaction between the two molecules, such as one note masking another, must be learned indirectly by the downstream network ρ from the aggregated representation. The Set Transformer, described next, is designed to model such interactions directly [41, 44].

Set Transformers and Attention-Based Pooling

Simply averaging or concatenating the embeddings of the two molecules, as described above, combines them through a fixed operation that does not depend on either molecule’s content. To let

the two embeddings influence one another, a Set Transformer allows each representation to be adjusted in light of its partner while remaining permutation-invariant [25]. The Set Transformer is composed of two blocks. The Self-Attention Block (SAB) lets the two molecule embeddings in a blend cross-attend to one another [25, 46], so that each representation is conditioned on its partner before pooling. This is designed to let the two embeddings influence each other—for example, letting the model learn that one molecule’s contribution changes in the presence of the other—rather than treating them independently as averaging does [33, 41]. Whether this mechanism leads to better predictions in practice is tested empirically in this study.

The second block is Pooling by Multihead Attention (PMA):

$$\text{PMA}_k(X) = \text{MAB}(S, \text{rFF}(X)). \quad (3.6)$$

This process begins by taking the molecular embeddings $\mathbf{X}$ and passing them through a row-wise feedforward network that prepares each molecule’s data for the final pooling calculations. The PMA block introduces a set of k learnable seed vectors, denoted as (S) . These vectors are refined during training to actively extract the dominant features of the scent blend. The Multihead Attention Block (MAB) lets these seed vectors (S) attend to the polished molecule embeddings $(\text{rFF}(X))$, compressing the set of two molecule embeddings into a fixed-size blend representation [25, 41].

3.1.5 Readout Expressivity Limits and Symmetry Collapse

The two chemical embeddings must be merged into a single representation before classification happens. Standard Graph Neural Network (GNN) designs are usually simple when merging vectors in which it typically use linear permutation-invariant pooling, such as sum or mean pooling [54]. This kind of pooling has a mathematical ceiling: many different pairs of embeddings can share the same average, so a linear readout cannot always distinguish them, regardless of how much structural detail the encoder captured beforehand [44]. This is a limit on what a linear readout can represent in the worst case, not a claim about how this specific model performs on this dataset [41]. This argument concerns the mixture-level readout, where two molecule embeddings are combined; the within-molecule mean-pooling readout in Level 1 follows common practice in molecular GNNs and is not the focus of this study.

Why Averaging Loses Information

Averaging two molecule embeddings can make structurally different pairs indistinguishable to the model. In one dimension, the pairs (2, 8), (4, 6), and (5, 5) all average to 5, so after averaging, the model cannot tell them apart no matter how precisely each molecule was encoded. Applied to blend prediction, two chemically different molecule pairs can average to the same blend representation, even when the GAT encoder produced embeddings that were genuinely sensitive to the difference between them. Concatenation avoids this specific collapse but is not permutation-invariant on its own, since its output depends on which molecule is listed first. This is the gap the Set Transformer is intended to close: letting the two embeddings interact before combining them, while remaining order-independent, unlike concatenation. A full representation-theoretic derivation of this argument, including its automorphism-group formulation, is given in Appendix A.1.

3.1.6 Label Reliability: Reciprocal Agreement, Cohen’s Kappa, and Noisy Labels

Because the blend labels used in this study are compiled from catalog recommendations rather than a controlled sensory panel, their reliability must be measured before model comparisons can be interpreted. Two complementary checks are used.

Reciprocal agreement is a within-catalog check. In the GoodScents catalog, molecule A’s page may list molecule B as a blending partner under a given scent word, but molecule B’s page may not list molecule A under the same word. When both pages agree, the label is *reciprocal*; when only one page makes the claim, the evidence is weaker. This study treats reciprocal agreement as one criterion for assigning a blend–descriptor pair to a higher reliability tier.

Cross-catalog agreement is measured using Cohen’s kappa (κ), which quantifies how consistently two independently maintained catalogs (GoodScents and Leffingwell) assign the same descriptor to the same molecule, correcting for the agreement expected by chance alone. A commonly used benchmark scale classifies κ values between 0.21 and 0.40 as “fair” agreement, 0.41 to 0.60 as “moderate,” and above 0.60 as “substantial” agreement [?]. These categories are used here as operational thresholds for the reliability analysis, not as universal or objective judgments of label correctness; recent methodological work cautions that kappa interpretation depends on the study conditions and should account for uncertainty. Descriptors whose cross-catalog κ falls below the

selected fair threshold are assigned to a lower reliability tier.

Learning from noisy labels. Even after tiering, some label noise remains. To test how each combination method responds to unreliable labels, the models are retrained on the high-confidence (Gold-tier) training partition with a controlled fraction of labels randomly flipped at 0%, 10%, 20%, and 30%. If the Set Transformer’s advantage over the baselines survives increasing noise, that suggests the advantage is not an artifact of label quality; if it disappears, the practical benefit depends on how clean the labels are [?].

3.2 Conceptual Framework

The conceptual framework of this study operationalizes the theoretical formulations into a systematic computational pipeline structured around the Input-Process-Output (IPO) model. Figure 3.1 outlines the structural interaction between input data representations, the two-level hierarchical deep learning architecture, and the downstream analytical and practical deliverables.

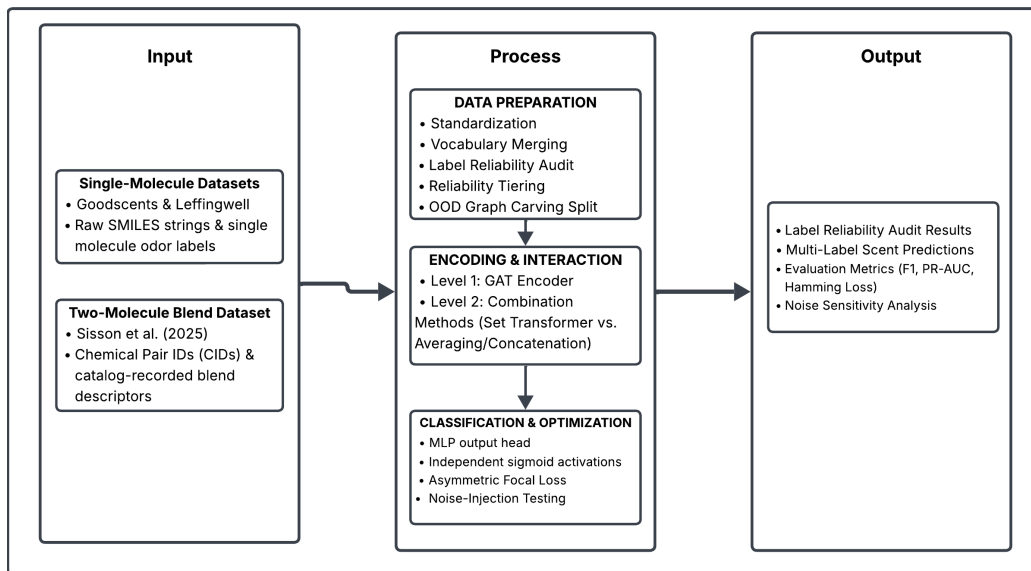


Figure 3.1: Input-Process-Output (IPO) Model.

3.2.1 Input Phase

The computational data pipeline starts with the acquisition, curation, and gathering of data from olfactory databases. The raw SMILES strings are extracted from the GoodScents and Leffingwell repositories accessed via the Pyrfume Library in Python [5]. These raw chemical blueprints are then paired with catalog-recorded odor tags to establish baseline inputs required from the pre-training phases. The binary mixture profiles are retrieved from the Sisson et al. (2025) benchmark dataset [41]. The retrieved benchmark dataset provides standardized chemical pair identifiers (CIDs) and multi-label scent descriptors that will help the network to learn and predict the combined scent of two scent blends.

3.2.2 Process Phase 1: Data Preparation and Graph Carving

To raw chemical input will first pass through a standardization pipeline. Salts and other non-fragrance components are removed using RDKit’s SaltRemover module, leaving the main molecule. Compounds containing elements other than C, H, O, N, S, F, Cl, Br, and I are dropped, since these fall outside the range typical of aroma chemicals [31]. The molecular weight should be between 20 and 600 g/mol. Below this range is considered irrelevant, while above the range is considered too heavy to be perceived as a scent at all [24]. The surviving SMILES strings are canonicalized using RDKit. The scent descriptors text from the source databases is standardized by merging redundant terms and eliminating descriptors that appear rarely to learn from [7]. The blend labels are then audited for reliability before any model is trained. Reciprocal agreement is measured for each blend pair, and cross-catalog agreement between GoodScents and Leffingwell is measured on the parent molecules using Cohen’s kappa. Each blend descriptor is assigned a Gold, Silver, or Bronze tier based on these two measurements, with the tier rules fixed in advance. To test how well the network handles unfamiliar compounds, molecules are grouped into structural families by UMAP and K-Means on 2048-bit Morgan fingerprints [13]. The Graph-Carving algorithm then divides the mixture meta-graph along family clusters. This guarantees that validation and testing pairs do not contain any molecule encountered during training [10].

3.2.3 Process Phase 2: Encoding and Interaction

The second phase converts the flat, two-dimensional 2D molecular graphs into continuous high-dimensional vector spaces and models non-linear interactions between paired compounds. In the encoder, individual atoms and bonds are processed as graph nodes and edges [52]. Since the Level 1 GAT shares its weights across both molecules, it ensures that the two are encoded under the same learned rules [39]. After encoding and updating atomic neighborhoods, global mean pooling condenses these vector spaces into a single 256-dimensional embedding for every compound [39, 52].

The two embeddings are then combined into a single representation using one of three methods, which are compared against each other mean pooling; concatenation, following the pairwise design of Sisson et al. [41]; or the Level 2 Set Transformer, in which each embedding is adjusted according to its molecular before the two are pooled into a single 256-dimensional vector [25]. The first two combine the embeddings through a fixed operation, while the Set Transformer lets the two molecules interact. Whether that interaction improves prediction is the central question of the study.

3.2.4 Process Phase 3: Classification and Optimization

Once the Level 2 yields the 256-dimensional blend vector spaces, it will then be translated into multi-label scent predictions. This Multi-Layer Perceptron (MLP) head contracts sequentially from 256 to 128 and ends with L individual outputs to match the target scent classes [41]. Because scent profiles are inherently multi-label, the classification layer utilizes independent sigmoid activation functions [37]. These activations compute individual probability scores for the joint presence or absence of each unique target scent descriptor. The system is trained end-to-end for network optimization. This means feedback from prediction errors flows backward through all layers simultaneously, fine-tuning both the Level 1 molecular encoders and the Level 2 mixture blocks guided with Asymmetric Focal Loss [16]. Each combination method is trained under two data conditions, on all reliability tiers and on Gold-tier blends only, and, for the noise sensitivity test, on Gold-tier blends with a controlled fraction of labels corrupted. Olfactory datasets are highly sparse, meaning a single scent profile contains only a few active descriptors alongside dozens of inactive ones (zeros). Under standard training rules, the model would quickly learn that it can achieve approximately 98% accuracy simply by predicting “no” for every label. The asymmetric loss solves this class imbalance by ignoring these obvious negatives and forcing the network’s mathematical focus onto the rare, active scent notes

that are difficult to predict [16].

3.2.5 Output Phase

The final phase outputs a multi-label binary vector of predictions across all L target scent descriptors which is based on the probability given to each descriptor. A validation module evaluates the model’s performance on these predictions under specified conditions. It follows and calculates metrics such as macro- and micro-averaged F1-scores, Hamming loss, and the Precision-Recall Area Under the Curve (PR-AUC) [3, 45, 48]. These metrics measure the model’s performance, ensuring that it generalizes well to new chemical structures. The three combination methods are compared on the high-confidence (Gold-tier) test set, and also on the full test set for transparency. Each method is trained on all reliability tiers and on Gold-tier blends only, and differences between methods are assessed using paired bootstrap confidence intervals. To test how each method responds to unreliable labels, the models are also retrained on Gold-tier blends with a controlled fraction of labels corrupted, and the decline in macro F1 is compared across methods.

The study additionally reports the results of the label reliability audit: reciprocal agreement rates, cross-catalog agreement by descriptor, and the number of blends and descriptors in each reliability tier.

3.3 Technical Background

Mapping molecular structure onto multi-label olfactory predictions requires stitching together a few different layers of infrastructure: cheminformatics standards to make sure the molecules are valid, a graph learning framework to process them, and a loss function suited to how sparse the labels are. This section walks through each piece—the parsing standards, the sparse graph representation, and the loss objective—and explains why each choice was necessary rather than incidental.

3.3.1 Cheminformatics Standard (RDKit): SMILES Parsing and Sanitization

Molecular structures in this study are represented as Simplified Molecular Input Line Entry System (SMILES) strings, a compact text-based notation, which are canonicalized and parsed into undirected, attributed molecular graphs using the open-source RDKit library in Python [31]. SMILES

strings are convenient because they are compact and suitable for storage. This compactness comes at a cost, however: they are structurally sensitive, prone to syntax errors, and do not encode explicit 3D geometry [49]. Before any structure reaches the model, every string passes through RDKit’s sanitization pipeline, which runs four checks in sequence:

- Kekulization checks that the alternating single and double bonds within conjugated or aromatic rings are represented in a chemically consistent manner, resolving ambiguous resonance structures.
- Aromaticity Perception evaluates the coordinate and topological structure of each atom to identify aromatic rings, dynamically reassigning aromatic bond attributes to relevant edges.
- Valence Checking checks every atom, specifically heavy atoms—carbon, nitrogen, oxygen, sulfur, and halogen to ensure it does not exceed its physical valency limits. This rejects syntactically corrupted SMILES strings or physically impossible hypervalent structures.
- Ring Perception, which identifies and indexes ring systems, information the later convolutions depend on.

Enforcing these rules ensures that what the network receives is already known to be chemically valid, rather than requiring the model to infer the rules of organic valency implicitly from training data [4, 56]. The sanitization pipeline acts as an explicit structural filter ahead of the model, so that the Level 1 Graph Attention Network (GAT) operates only on topologically valid graphs that correspond to real molecular structures [24, 52].

3.3.2 Deep Learning Stack (PyTorch / PyTorch Geometric): Sparse Graph Convolutions

After sanitization, each molecule is formatted as an undirected graph $G = (V, E)$, where atoms are nodes (V) and covalent bonds are edges (E). A dense $N \times N$ adjacency matrix could represent this connectivity, but for molecules, where most atoms bond to only a few neighbors, this wastes memory, scaling at $\mathcal{O}(N^2)$ for structures that are inherently sparse. The architecture instead runs on PyTorch Geometric, which handles molecular graphs as sparse attributed objects rather than dense arrays [?].

Topology is stored in Coordinate (COO) format: a sparse two-row index matrix C of size $2 \times 2E$, where the first row lists source node indices and the second lists receiver node indices [4]. Since chemical bonds are physical and undirected, every source node is also a receiver, so $2E$ entries are needed rather than E — each bond is represented in both directions so that message-passing can update both of its atoms. Node states are updated by running the GAT computation described in Section 3.1.3 over this sparse index matrix, rather than over a dense adjacency matrix, at each layer l .

3.3.3 Multi-Label Mathematical Formulation:

Independent Sigmoids and Sparsity Mitigation

A molecule or blend can carry several scent descriptors at once (for example, both fruity and green), rather than belonging to a single exclusive category. To reflect this, the final layer skips a mutually exclusive softmax and instead maps the 256-dimensional blend embedding through an MLP to L independent sigmoid outputs, one per target scent descriptor [37]. Each output is a probability $y_{\text{pred},k}$ bounded between 0 and 1:

$$y_{\text{pred},k} = \sigma(x_k) = \frac{1}{1 + e^{-x_k}}. \quad (3.7)$$

A standard multi-label setup would optimize this with plain Binary Cross-Entropy, summing error across all L target classes independently:

$$\mathcal{L}_{\text{BCE}} = -\frac{1}{L} \sum_{k=1}^L \left[y_k \log(y_{\text{pred},k}) + (1 - y_k) \log(1 - y_{\text{pred},k}) \right]. \quad (3.8)$$

Olfactory data is severely imbalanced, however. In the dataset used in this study, label density is approximately 2%, meaning a typical blend has only a few active descriptors out of the full vocabulary, with the remainder absent. Under plain BCE, this large majority of easy negatives dominates the gradient — a model can achieve high overall accuracy simply by predicting zero for every descriptor, and in doing so never learns the rare positive labels that are actually informative.

To address this, the pipeline uses an Asymmetric Focal Loss, which splits the focusing behavior for positives and negatives into separate exponents, γ_p and γ_n , and adds a class-frequency weight α_k [16]:

$$\mathcal{L}_{\text{FL}} = -\frac{1}{L} \sum_{k=1}^L \left[y_k \cdot \alpha_k \cdot (1 - y_{\text{pred},k})^{\gamma_p} \cdot \log(y_{\text{pred},k}) + (1 - y_k) \cdot (1 - \alpha_k) \cdot y_{\text{pred},k}^{\gamma_n} \cdot \log(1 - y_{\text{pred},k}) \right]. \quad (3.9)$$

Setting $\gamma_p = 0$ leaves gradients from the rare positive labels untouched — they pass through at full strength. Setting $\gamma_n = 2$ does the opposite job for negatives: it exponentially suppresses the loss contribution from easy, correctly predicted zeros, stripping away their gradient dominance. Together, these two settings push the model’s training signal toward the rare positive notes instead of allowing it to coast on negatives, which is what makes multi-label prediction feasible on data this sparse [16].

Chapter 4

Methodology

This chapter describes the preparation of the catalog-derived datasets, the construction of reliability masks, the formation of component-disjoint data partitions, and the matched training and evaluation of three aggregation methods. It also defines the robustness conditions, metrics, and paired statistical comparisons used to answer the research questions. Because this is a proposal, the procedures are written in the future tense. Settings that have not yet been approved, and numbers that can only be known after the final pipeline is run, are shown as symbols or marked as pending; none of them are results.

4.1 Research Design

This study will use a quantitative experimental design on a multi-label molecular graph classification task. It compares three ways of combining the learned embeddings of the two molecules in a blend: mean pooling, order-averaged concatenation, and Set Transformer aggregation. Fragrance-blend prediction is the application. The computer-science investigation concerns how an unordered pair of graph embeddings should be aggregated, how uncertain labels should be handled, and how models should be evaluated when the two molecules of a pair may also appear in other pairs.

The research questions are the following.

RQ1. Under matched encoding, training, and evaluation conditions, how does the performance of Set Transformer aggregation compare with mean pooling and order-averaged concatenation on

the reliability-screened test data?

RQ2. How does the performance of Set Transformer aggregation relative to the two baselines vary across additive, catalog-label loss, and catalog-label gain patterns?

RQ3. How does the robustness of Set Transformer aggregation compare with the two baselines as training-label reliability decreases, including under controlled label corruption?

Table 4.1 states what is varied and what is held fixed. The aggregation method is the primary varied factor. Label-reliability and label-corruption conditions are robustness factors, and the catalog-label pattern of a blend is used only for a secondary subgroup analysis. Everything else is a matched control, so that a difference between aggregation methods is not caused by a difference in data, training budget, or luck of the seed.

Table 4.1: Varied factors and matched controls.

Category	Treatment
Aggregation method	Primary varied factor (three levels)
Label-reliability and corruption conditions	Robustness factors
Catalog-label pattern	Secondary subgroup analysis
Encoder design and initialization policy	Matched control
Train/validation/test partitions	Fixed across variants
Loss, optimizer, stopping rule, training budget	Matched controls
Random seeds and corrupted datasets	Paired across variants

The dependent variables are macro F1 (the primary metric, declared before any experiment is run), micro F1, PR-AUC, and Hamming loss, defined in Section 4.7.

4.2 Data Acquisition and Preparation

This section describes the data sources, how they are joined, and how the model inputs (molecular structures) and the model targets (odor descriptor labels) are prepared. The two are prepared separately: structures become graphs, and descriptors become a fixed list of labels.

4.2.1 Data Sources and Provenance

The study will use three sources. The single-molecule sources are GoodScents and Leffingwell, both accessed through Pyrfume. They are independently maintained reference catalogs for the perfumery and flavor industries. Each record lists a molecule, its structure, and the scent descriptors assigned to it (for example, fruity or sweet). The Leffingwell descriptors are described as assigned by olfactory experts, usually practicing perfumers [24]. The blend source is the two-molecule dataset of Sisson et al. [41], obtained from their public repository.

Each Pyrfume source has three files: `behavior.csv`, which holds stimulus–descriptor pairs; `stimuli.csv`, which holds the stimulus identifier, the GoodScents or Leffingwell identifier, the PubChem CID, concentration, and solvent; and `molecules.csv`, which holds the CID, molecular weight, isomeric SMILES, IUPAC name, and name. Descriptors in `behavior.csv` are indexed only by stimulus, so they will first be joined to `stimuli.csv` on the stimulus field to obtain a CID, and then to `molecules.csv` on the CID. Concentration and solvent columns will be dropped, because concentration is outside the scope of this study.

Structure filtering. Before molecules are matched across sources, structures will be sanitized with RDKit: salts are removed, molecules containing elements other than C, H, O, N, S, F, Cl, Br, and I are excluded, and molecules outside a molecular-weight range are excluded. **[PENDING: confirm the element list and the molecular-weight range (20–600 g/mol in the earlier pipeline) and the resulting molecule count, which is a result of the final run and is not stated here.]** This step uses structure only; no label is consulted.

Join key. The two single-molecule catalogs will be merged into one master dictionary using the isomeric SMILES string as the key. Every SMILES string in both sources, and in the blend dataset, will be canonicalized with RDKit so that the same molecule written in two different ways is matched correctly. SMILES was chosen over the PubChem CID because some CIDs in the source files are negative or non-standard and would need further standardization, whereas SMILES must be canonicalized anyway when molecules are converted to graphs. Blend components in the Sisson et al. dataset, which are identified by CID, will be linked to the master dictionary by canonicalizing their SMILES in the same way.

Blend records. The blend dataset of Sisson et al. was built by crawling the “Potential Blenders” section of each GoodScents molecule page, which lists the molecules that molecule could be paired with under scent headings. The labels of a pair are the headings under which the partner was listed. **[CONFIRM: re-read the Sisson et al. Methods and supplement and state exactly how a heading becomes a blend label before this sentence is final.]** Because each molecule’s page was crawled separately, a pair can be listed from both molecules’ pages. The pair-building code of Sisson et al. counts how many pages listed a pair under each heading (one or two) but does not keep the count. This study will keep that count, because it provides the reciprocal evidence used in Section 4.3.

4.2.2 Ground Truth and Its Limits

Catalog-recorded descriptors serve as proxy targets for supervised learning. Reliability procedures estimate the consistency of those records; they do not establish perceptual correctness. The catalogs are curated reference records, not live sensory panel measurements, and their descriptors come from predefined word lists without a reported procedure for combining individual judgments. They therefore contain some noise and subjectivity [?]. This study treats them as a practical proxy for perceived scent, estimates how consistent they are, and examines how model comparisons change as label reliability changes.

For the computational scope of this study, each blend is treated as a simple physical mixture of two aroma chemicals. No significant intermolecular reaction is expected for most pairs under the assumed blending conditions (room temperature, no catalyst), so the data are treated as standard fragrance blending rather than reaction chemistry. A few highly reactive pairs may deviate from this expectation. **[CONFIRM: wording comes from chemistry faculty; permission to name the source is pending.]**

4.2.3 Target Vocabulary

Two vocabularies must be kept apart. The *standardized vocabulary* is the list of descriptor names after spelling variants have been merged. The *final target vocabulary*, of size L , is the subset of the standardized vocabulary on which models are actually trained and scored.

Standardization. The raw GoodScents and Leffingwell labels contain spelling variants of the same descriptor (for example, anisic and anise, wine and winey, or caramel and caramelic). Reviewing every label by hand would be impractical, so the study will adopt the open-source canonicalization dictionary of Sisson et al. [41], which merges raw blend descriptors into 104 standardized descriptors. **[CONFIRM: the source text gives both “approximately 400 raw descriptors” and “109 raw notes” for the input to this step; check which is correct and use one.]** Any descriptor in the merged data that the dictionary does not cover will be reviewed manually and either mapped to an existing descriptor or declared new, followed by whitespace and punctuation normalization. The standardized single-molecule vocabulary may exceed 104 descriptors if new descriptors survive this review; Appendix B will list it.

Final target vocabulary. The final target vocabulary will be built from *training data only* (Section 4.5.4). Validation and test labels will not be consulted when deciding which descriptors are kept. The size L is therefore not known yet and is written as a symbol throughout. The figure 104 is the size of the standardized vocabulary before partitioning, and the figure 74 is the number Sisson et al. reported after their own partitioning; neither is assumed to be this study’s L .

Inputs and targets. Input preparation (canonical SMILES to molecular graph with atom and bond features, Section 4.4.1) uses only molecular structure. Target preparation (descriptor standardization, mask construction, vocabulary selection) uses only descriptor labels. Neither step uses the test partition to make a decision.

4.2.4 Label Families (Reporting Only)

Expert-curated odor taxonomies, such as that of Sajan et al. [38], group raw odor words into broader families (for example, rose, jasmine, and violet under floral). The model will still predict each descriptor individually, because catalog-label loss and gain refer to specific words rather than to families. The Sajan et al. grouping is used for organized reporting only: Appendix B will map each descriptor to a family and give the number of descriptors per family. No per-family scores will be computed.

4.3 Label Reliability Mask

Not every recorded label is equally trustworthy, and not every missing label is a true absence. Rather than assigning one reliability grade to a whole blend, the study will decide, for each *blend-descriptor cell*, whether that cell is eligible for a stricter, “Gold-only” treatment. A cell is one entry of the blend-by-descriptor label matrix: one blend and one descriptor.

4.3.1 Consistency Evidence

Two kinds of evidence are used. Both measure consistency, not correctness.

- **Reciprocal-positive evidence (cell level).** A recorded positive is *reciprocal* if the pair was listed under that descriptor from the pages of both molecules (a count of two). Using the pair-building code of Sisson et al. [41], a preliminary audit found that 61,838 of 235,040 listings (26.3%) were reciprocal. **[CONFIRM: this is a preliminary audit figure; recompute with the final pipeline before reporting.]** A one-sided listing is not necessarily wrong, since catalogs do not list every possible blend, but it is not independently confirmed.
- **Descriptor-level consistency (descriptor level).** Agreement between GoodScents and Leffingwell on the molecules they share will be measured separately for each descriptor using Cohen’s kappa. In a preliminary audit on 2,225 shared molecules, 64 of 104 descriptors reached a kappa of at least 0.21 (“fair” on the scale of Landis and Koch). **[PENDING: the statistic and the cutoff are not final. The team must approve the statistic, the cutoff, and the molecule set used to compute it.]** A descriptor that meets the cutoff is called *consistent*. Reaching the cutoff does not prove that the labels are correct.

4.3.2 Eligibility Mask

Table 4.2 defines the eligibility mask $M_{ij} \in \{0, 1\}$ for blend i and descriptor j in the Gold-only treatment.

Because eligible unlisted descriptors enter as negatives, a Gold-only test set is not positive-only: a model that predicts a descriptor that is not recorded for a blend is still penalized as a false positive.

Closed-world assumption. Treating an unlisted descriptor as a negative is an operational assumption. It is needed because supervised multi-label learning requires a definition of “absent.”

Table 4.2: Eligibility of a blend–descriptor cell in the Gold-only treatment.

Cell evidence	Descriptor is consistent	Gold-only treatment
Reciprocal recorded positive	Yes	Include as positive ($M_{ij} = 1$)
One-sided or otherwise ambiguous recorded positive	Either	Mask ($M_{ij} = 0$)
Unlisted descriptor	Yes	Include as negative ($M_{ij} = 1$), under the closed-world assumption
Any cell	No	Mask ($M_{ij} = 0$)

Catalog omission may not mean true sensory absence, since a catalog may simply not list a blend or a descriptor. The reliability mask reduces this uncertainty but does not remove it, and results will be interpreted with this limitation in mind.

4.3.3 Experimental Label Conditions

Two label conditions are used.

- **Full-label condition.** Every cell is used ($M_{ij} = 1$ for all i, j), with listed descriptors as positives and unlisted descriptors as negatives.
- **Gold-only condition.** Only cells with $M_{ij} = 1$ under Table 4.2 are used.

Masked loss. In the Gold-only condition a blend is kept as long as it has at least one eligible cell; it is not removed merely because some of its cells are ambiguous. Loss is computed only over eligible cells:

$$\mathcal{L}_{\text{masked}} = \frac{\sum_i \sum_j M_{ij} \ell_{ij}}{\sum_i \sum_j M_{ij}} \quad (4.1)$$

where ℓ_{ij} is the per-cell loss (Section 4.4.3). In the full-label condition $M_{ij} = 1$ everywhere and the expression reduces to the ordinary mean loss. The same mask is applied when Gold-only metrics are computed (Section 4.7). The training, validation, and test partitions each carry their own mask, built from the same rule.

Insufficient support. After masking, a descriptor may have no eligible positives, or no eligible negatives, in a partition. F1 and PR-AUC are undefined for such a descriptor. Any descriptor with fewer than s eligible positives or fewer than s eligible negatives in the evaluated partition will

be excluded from macro-averaged scores, and the excluded descriptors will be listed in the report. **[PENDING: the minimum support s must be fixed before final evaluation.]** A training run that has no eligible cells at all is reported as a failed run and is not silently dropped.

4.4 Model Architecture

All three models share one molecular graph encoder and one output head. They differ in the step that combines the two molecule embeddings. The data flow is: each molecule is turned into a graph and encoded into a vector; the two vectors are combined by the aggregation method; and the combined vector is converted into one probability per descriptor.

4.4.1 Shared Graph Encoder

Each molecule is represented as a graph in which atoms are nodes and bonds are edges. A Graph Attention Network (GAT) encoder turns each graph into a fixed-length vector $\mathbf{z} \in \mathbb{R}^d$, so a blend yields two embeddings $\mathbf{z}_A$ and $\mathbf{z}_B$. The same encoder, with the same weights, is applied to both molecules.

[CONFIRM: node feature size, edge feature size, number of GAT layers, attention heads, readout, and embedding size d must be copied from Chapter 3 after the revised architecture is approved. The values used in the earlier notebook (46 node features, 10 edge features, 3 layers, $d = 256$) are not inherited automatically.] The attention computation is defined in Chapter 3 **[TODO: replace with \ref to the Chapter 3 section label]**. In the rest of this chapter d stands for the embedding size and, where a concrete number is needed for illustration, $d = 256$ is used.

Pretraining. The encoder may first be pretrained on single-molecule data, using a temporary head with one sigmoid output per standardized single-molecule descriptor and binary cross-entropy loss. Only molecules assigned to the training partition (Section 4.5) will be used, so molecules reserved for validation and testing are never seen during pretraining. After pretraining the temporary head is discarded and only the encoder weights are kept.

Encoder policy. **[PENDING: the team must choose one policy for the final comparison.]** The earlier notebook’s freeze/unfreeze schedule is implementation experience only and is not

the revised protocol. The two options are:

- a *frozen shared encoder*, in which encoder weights are fixed after pretraining and only the aggregation module and output head are trained; or
- the *same pretrained initialization followed by matched end-to-end fine-tuning* for all variants.

If the second option is chosen, the text will state: “The variants begin from the same encoder initialization and use matched schedules and optimization settings. Their encoder weights may diverge during end-to-end training.” In that case this study will not claim that the final models differ only in how they combine embeddings, because their encoders can end up different. Either way, the initialization policy, schedule, and training budget will be identical across the three aggregation methods.

4.4.2 Aggregation Variants

Table 4.3 summarizes the three variants. For two molecules, the embeddings form an unordered pair $\{\mathbf{z}_A, \mathbf{z}_B\}$, and a blend prediction should not change if the two molecules are swapped (permutation invariance).

Table 4.3: The three aggregation variants.

Variant	General operation	Order handling
Mean pooling	Average the two molecule embeddings	Invariant by construction
Order-averaged concatenation	Run both ordered concatenations through the head and average the outputs	Explicit symmetrization
Set Transformer aggregation	Apply set attention, then invariant pooling	Invariant set processing

Mean pooling. The blend embedding is the unweighted average of the two molecule embeddings,

$$\mathbf{z}_{\text{blend}} = \frac{1}{2}(\mathbf{z}_A + \mathbf{z}_B) \in \mathbb{R}^d. \quad (4.2)$$

This is simple and invariant to order. Sum pooling is not included as a separate variant: for exactly two molecules the sum is twice the mean, so it adds no information once the output head is trainable.

Order-averaged concatenation. The two embeddings are placed side by side, $[\mathbf{z}_A | \mathbf{z}_B] \in \mathbb{R}^{2d}$ (512 dimensions when $d = 256$). Raw concatenation depends on the order of its inputs, so the model is evaluated on both (A, B) and (B, A) and the two outputs are averaged. This follows the pairwise design of Sisson et al. [41], but it is an *adapted baseline*, not a reproduction of their whole model: it uses this study’s encoder, loss, and partitions. **[PENDING: state whether the averaging is applied to logits or to probabilities, and use the same choice in every chapter and slide. Averaging logits and applying the sigmoid once is the simpler option, but the team must decide.]**

Set Transformer aggregation. The pair $\{\mathbf{z}_A, \mathbf{z}_B\}$ is treated as a set and passed through a Set Attention Block (SAB), in which each embedding attends to both embeddings in the set (itself included):

$$\text{SAB}(X) = \text{MultiHead}(X, X, X), \quad (4.3)$$

where queries, keys, and values all come from the same set X . This lets the representation of each molecule be adjusted using the other before the two are combined. The attention mechanism follows the definition in Chapter 3 **[TODO: \ref]**. The adjusted pair is then reduced to a single blend embedding $\mathbf{z}_{\text{blend}} \in \mathbb{R}^d$ by Pooling by Multihead Attention (PMA), which uses learned attention weights instead of a plain average. **[PENDING: number of SAB blocks, number of attention heads, hidden size, and whether PMA uses one seed vector; values from the earlier notebook (2 blocks, 4 heads) are not inherited automatically.]** Both SAB and PMA are permutation-invariant in the sense needed here: swapping $\mathbf{z}_A$ and $\mathbf{z}_B$ does not change $\mathbf{z}_{\text{blend}}$. This module is a learned way of mixing two embeddings. This study does not claim that it models physical or receptor-level interaction between the molecules.

Complexity. Once the models are implemented, the study will report the number of trainable parameters of each full model and a measure of computation cost (for example, training time per epoch and inference time per batch on the same hardware), so that any accuracy difference can be read together with the cost of obtaining it. **[FILL IN: parameter counts and timings.]**

4.4.3 Output Head and Loss

The blend embedding is passed through a Multi-Layer Perceptron that maps $d \rightarrow h \rightarrow L$, where h is a hidden width and L is the final target vocabulary size (Section 4.2.3). **[PENDING: h and L ; the earlier notebook used $h = 128$.]** For order-averaged concatenation the first layer takes $2d$ inputs. The output layer applies an independent sigmoid to each of the L nodes, because odor descriptors are multi-label: each descriptor is its own present/absent decision, unlike softmax, which forces the outputs to sum to one. The framework for multi-label learning and the Asymmetric Focal Loss are defined in Chapter 3 **[TODO: \ref]**. Following Hatami and Moradi [16], the loss uses a focusing parameter of $\gamma_p = 0$ for positives and $\gamma_n = 2$ for negatives **[PENDING: confirm these values for the revised study]**. The per-cell loss ℓ_{ij} of this function enters the masked loss of Section 4.3. All three variants use the same loss, optimizer, learning-rate schedule, early-stopping rule, and training budget.

4.5 Component-Disjoint Partitioning and Graph Carving

This section describes how the data are divided into training, validation, and test sets. The aim is that no molecule that appears in a training blend also appears in a test blend. This section describes the procedure; it does not claim that the test set contains only chemically unfamiliar molecules.

Why random pair splitting leaks components. A blend dataset is a collection of pairs, and the same molecule appears in many pairs. If pairs are split at random, a molecule seen in a training pair is very likely to appear again in a test pair, so test performance partly measures memorization of molecules rather than prediction for new ones. Group-aware splitting is the standard remedy for this kind of overlap [21] (see Section 2.13).

4.5.1 Scaffold Split Decay

Scaffold splitting, which groups molecules by shared core framework, is often used for this purpose. In sparse chemical spaces, however, most scaffold groups contain very few molecules, and scaffold splitting then behaves much like random splitting (Section 2.13). To check this on the present data,

the molecule-to-scaffold ratio will be computed:

$$R = \frac{N_{\text{molecules}}}{N_{\text{scaffolds}}} \quad (4.4)$$

where $N_{\text{molecules}}$ is the number of unique canonical molecules and $N_{\text{scaffolds}}$ is the number of distinct Bemis–Murcko frameworks produced by RDKit. A value of R near 1 means almost every scaffold holds a single molecule. R will be computed on the merged master dictionary and compared with the collapse threshold $R < 2.0$ reported by Fooladi et al. [10]. **[CONFIRM: check this citation key against references.bib.]**

4.5.2 Cluster-Based Partitions

Molecules will be grouped by structural similarity and whole groups will be assigned to one partition. The planned pipeline is: compute a binary structural fingerprint for every unique molecule; reduce it to a low-dimensional map with UMAP; and cluster the map with K-Means into $k = 7$ clusters. The number of clusters is fixed at seven, following Guo et al. [13], who found seven UMAP clusters to outperform alternative clustering methods on molecular data. A candidate fingerprint is ECFP4 (2048 bits, radius 2).

[PENDING: fingerprint type and size, UMAP settings, random seeds, and the cluster-to-partition rule are not final. They will be fixed and written down before any final evaluation, and the values of the earlier notebook will be reused only if the team deliberately re-approves them. The value $k = 7$ is the only clustering setting treated as fixed.]

The rule that assigns clusters to training, validation, and test partitions will be *declared in advance and will not depend on any label*. For example, it may depend only on cluster sizes (such as assigning clusters to partitions so that the molecule counts approximate a target ratio). **[PENDING: state the exact rule.]** The selection will not look at which descriptors occur in which cluster, and it will not look at test labels. Once the partitions are fixed they are not changed; the number of molecules, pairs, and descriptor support in each partition are reported afterwards as outcomes, not used as selection criteria. If any inspection of test labels turns out to be unavoidable, it will be disclosed, and the test set will not be described as untouched.

4.5.3 The Graph-Carving Protocol

The blend dataset is treated as a graph in which each node is a unique canonical molecule and each edge is a documented blend between two molecules. Every molecule is assigned to a partition by its cluster. Carving then examines each edge: an edge (blend) is *retained only if both molecules belong to the same assigned partition*, and discarded otherwise. After carving, no molecule is shared between any two partitions, so no molecule component can leak from training into validation or test. Sisson et al. [41] applied a similar carving to a random 50:50 molecule partition, which discarded about 83,000 pairs and left about 44,000 training and 40,000 testing pairs. This study uses clusters instead of a random molecule split, so its pair counts will differ and are reported only after the procedure is run. **[FILL IN: pairs retained and discarded per partition.]**

The validation set is used for early stopping, hyperparameter decisions, and selection of decision thresholds. The test set is kept sealed and used only in the final evaluation.

Component-disjoint partitioning reduces component and structural overlap between partitions. It does not guarantee that test molecules belong to unfamiliar chemical families, because clusters that are near each other on the map can still contain similar molecules.

4.5.4 Training-Only Vocabulary Filter

Carving changes how often each descriptor occurs, so the final target vocabulary is chosen after partitioning, from training data only. A standardized descriptor d is kept if it has enough positive examples to learn from:

$$d \in \mathcal{D}_{\text{target}} \iff c_{\text{train}}(d) \geq x \quad (4.5)$$

where $c_{\text{train}}(d)$ is the number of training blends in which d is recorded as positive, and x is a minimum-support threshold. **[PENDING: the value of x ; the earlier notebook used $x = 5$ on both training and test sides, and the test-side condition is removed here.]** The size of $\mathcal{D}_{\text{target}}$ is L . In the Gold-only condition, $c_{\text{train}}(d)$ counts eligible positives. After the vocabulary is locked, the number of validation and test positives and negatives for each descriptor is *reported* but is not used to add or remove descriptors. Descriptors that end up with too little test support are handled by the insufficient-support rule of Section 4.3 and are listed in the report.

4.6 Experiments

All experiments hold the partitions, the encoder initialization policy, the loss, the optimizer, the stopping rule, the training budget, and the metric suite constant across the three aggregation methods. They will show whether attention-based aggregation performs better than simpler alternatives; a finding that a simpler baseline performs as well or better is a valid and informative result and is reported as such. Each configuration is run with the same set of random seeds for every aggregation method. **[PENDING: the number of seeds.]**

4.6.1 Experiments A and B: Main Comparison (RQ1)

Experiment A trains each model in the full-label condition, using all cells. Experiment B trains each model in the Gold-only condition, using masked loss over eligible cells (Section 4.3). Both are evaluated on the held-out test partition. Experiment A is also scored on all cells, and Experiment B is scored on eligible cells with the same mask. Reporting both makes visible how much of any difference depends on which cells are counted.

4.6.2 Experiment C: Controlled Label Corruption (RQ3)

To examine how the comparison changes as training labels become less reliable, labels will be corrupted on purpose, in the training partition only. Validation and test labels are never changed.

For a noise level $p \in \{0, 0.10, 0.20, 0.30\}$, a fraction p of the eligible positive cells in the training labels is flipped to absent, and an equal number of eligible negative cells is flipped to present, which keeps the density of positives unchanged. Only eligible cells can be flipped, so the same mask logic as Section 4.3 applies. For each combination of noise level and random seed, one corruption mask is generated and *reused across all three aggregation methods*; a new mask is generated for every seed and noise level. The run at $p = 0$ is the uncorrupted Gold-only run of Experiment B and is reused. Models are retrained at each level and scored on the unchanged validation and test labels. The change in macro F1 from $p = 0$ to $p = 0.30$ is then compared across methods using the paired bootstrap of Section 4.8.

Random flipping tests robustness to label errors in general. It does not reproduce every real process by which catalog errors arise.

4.6.3 Blend-Type Analysis (RQ2)

To see whether the relative performance of the methods differs by catalog-label pattern, each test blend is compared with the recorded descriptors of its two parent molecules. The terms below describe patterns in the catalog records; they are not claims about what a person would perceive.

- **Catalog-label loss:** a descriptor recorded for either parent is not recorded for the blend.
- **Catalog-label gain:** the blend has a recorded descriptor that is not recorded for either parent.
- **Additive:** the blend shows neither loss nor gain.

Descriptive statistics of the label matrix (label density, common co-occurrences, and the rate at which a blend’s labels are additive with respect to its parents) are reported for the dataset as a whole. They are used to describe the data and to support this subgroup analysis; they are not used to filter data. The sizes of these three groups are reported and the model comparison is repeated within each group. Because these groups overlap in how they are defined and the analysis is split into subgroups, it is treated as a secondary subgroup and error analysis rather than a confirmatory test. If any group contains fewer than 5% or more than 95% of the test blends, blends are instead divided by the overlap between the blend descriptors and the union of the parents’ descriptors.

4.7 Evaluation Metrics

This section defines the metrics and explains why each is needed when most label cells are negative.

4.7.1 Multi-Label Metrics

Let TP be a descriptor correctly predicted present, FP a descriptor incorrectly predicted present, FN a present descriptor that was missed, and TN a descriptor correctly predicted absent. Most cells in the label matrix are negative, so a model can look accurate by predicting “absent” every time. Precision, recall, and F_1 use only TP, FP, and FN [34], so they ignore the many easy true negatives, and a model that predicts nothing receives $F_1 = 0$.

- **Macro F1 (primary metric).** F_1 is computed for each descriptor separately and then averaged with equal weight, so rare descriptors count as much as common ones [34]. This choice

is declared before experimentation and must be the same in Chapters 1–4. [**CONFIRM: Chapter 1.**]

- **Micro F1.** TP, FP, and FN are pooled across all descriptors before F_1 is computed; it summarizes overall performance and favors common descriptors.
- **PR-AUC.** The area under the precision–recall curve, which measures how well the model ranks true descriptors above false ones across all thresholds. It is reported both macro-averaged over descriptors (primary version) and micro-averaged. [**PENDING: confirm that both versions are reported.**]
- **Hamming loss.** The fraction of evaluated cells that are predicted incorrectly (below).

Thresholds. Each descriptor’s probability is turned into a present/absent prediction with a decision threshold. Thresholds are chosen on validation data only and then applied unchanged to the test data. Threshold selection matters a good deal for imbalanced multi-label problems [48]. [**CONFIRM: check citation key; PENDING: whether one global threshold or a per-descriptor threshold is used.**]

4.7.2 Masked Metrics

All metrics are computed only over cells with $M_{ij} = 1$, using the mask of the condition being evaluated (all ones in the full-label condition). For macro metrics, descriptors with insufficient eligible support are excluded and listed (Section 4.3). With a mask, Hamming loss is

$$\text{HL}_M = \frac{\sum_{i=1}^N \sum_{j=1}^L M_{ij} \mathbf{1}(y_{ij} \neq \hat{y}_{ij})}{\sum_{i=1}^N \sum_{j=1}^L M_{ij}} \quad (4.6)$$

where N is the number of evaluated blends, L is the number of target descriptors, y_{ij} is the label, $\hat{y}_{ij} \in \{0, 1\}$ is the thresholded prediction, and $\mathbf{1}(\cdot)$ equals 1 if its argument is true and 0 otherwise. Smaller values are better, with $\text{HL}_M = 0$ for a perfect match. In the full-label condition $M_{ij} = 1$ everywhere and this is the ordinary Hamming loss.

All-negative reference. Hamming loss also rewards the many easy negatives, so it cannot be read alone. The *evaluated-cell density* is

$$D_M = \frac{\sum_i \sum_j M_{ij} y_{ij}}{\sum_i \sum_j M_{ij}} \quad (4.7)$$

the fraction of evaluated cells that are positive. A model that predicts every cell absent has $HL_M = D_M$, which serves as the all-negative reference. It is computed separately for each partition (D_{train} , D_{val} , D_{test}) and for each mask, and a value from one setting is never reused in another. **[FILL IN: densities and the all-negative reference, after the pipeline is run.]** Every model’s Hamming loss is reported next to this reference.

4.8 Statistical Comparison

Differences between models could reflect chance in which blends happen to be in the test set, or chance in training. These two sources are handled separately.

Variability across seeds. Optimization variability comes from random initialization and data order. Each model is trained with the same set of random seeds, and results are summarized across seeds (mean and spread) separately from the bootstrap below.

Bootstrap over blends. Descriptor outcomes within one blend are correlated, so individual cells are not independent. The test blends are therefore resampled as whole rows: a sampled blend keeps all of its descriptor cells together, with its mask. Naive paired t -tests are not appropriate for comparing machine-learning models, because they are sensitive to outliers and not valid when resampled test sets are used [34], and bootstrap resampling shows both whether a difference exists and how large it is [?].

From the N test blends, N rows are drawn with replacement, B times **[PENDING: the number of replicates B will be fixed before final evaluation]**. All models being compared are scored on the *same* resampled indices, giving a paired comparison. For replicate b the paired difference is

$$\Delta(b) = \text{Metric}_{\text{model 1}}(b) - \text{Metric}_{\text{model 2}}(b). \quad (4.8)$$

The direction of a difference is stated with each result: higher is better for macro and micro F1 and for PR-AUC, and lower is better for Hamming loss, so a positive Δ favors model 1 for the first three and model 2 for Hamming loss. The comparison is repeated for each metric and for each pair of aggregation methods (three pairs). **[PENDING: whether a correction for multiple comparisons is applied, or whether the primary comparison is limited to macro F1.]**

Intervals and effect size. The significance level is $\alpha = 0.05$. A 95% percentile confidence interval is taken from the 2.5th and 97.5th percentiles of the B differences, and an interval that excludes zero is read as a difference at $\alpha = 0.05$. Because statistical significance does not show whether a difference matters in practice, the effect size is reported as the mean difference with its 95% interval, in the units of each metric. For Experiment C, the bootstrap is applied to the change in macro F1 between noise levels.

4.9 Reporting Conventions

Experiments that have not been run are described in the future tense. Any pipeline demonstration run before the final protocol is fixed will be labelled preliminary and will not be reported as a result. Output from the earlier notebook is not reported as a result of the revised study. The study does not assume that the Set Transformer will outperform the baselines; if mean pooling or order-averaged concatenation matches or exceeds it, that outcome will be reported and discussed on equal footing.

Appendix A

Chemical Feature Vectors

The exact composition of the 46-dimensional node feature vector and the 10-dimensional edge feature vector used for the intra-molecular GAT encoder are detailed below.

Chemical Property	Dimensions	Mathematical / One-Hot Representation & Details
Atomic Identity	9	One-hot encoding of key organic and halogen elements in flavor/fragrance compounds: [C, N, O, S, F, Cl, Br, I, Other].
Formal Charge	1	Scalar representation of the localized net electrical charge on the atom.
Atomic Degree	6	One-hot representation of the number of connected covalent bonds to heavy atoms: [0, 1, 2, 3, 4, 5+].
Explicit Valence	7	One-hot encoding of the total valence electrons committed to bonding in the molecule: [0, 1, 2, 3, 4, 5, 6+].
Hybridization State	6	One-hot representation of the orbital configuration: [sp , sp^2 , sp^3 , sp^3d , sp^3d^2 , Other].
Aromaticity	1	Binary flag indicating if the atom is part of a resonance-stabilized aromatic ring structure.
Attached Hydrogens	5	One-hot encoding of the number of hydrogens bound to this heavy atom: [0, 1, 2, 3, 4+].
Ring Size Participation	6	Binary flags representing specific ring sizes to map cyclic functional groups: [3-ring, 4-ring, 5-ring, 6-ring, 7-ring, 8-ring].
Ring Membership	1	Binary flag indicating whether the atom is enclosed within any ring structure.
Chirality Tag	4	One-hot representation of tetrahedral stereocenter configuration: [Unspecified, Clockwise (CW), Counterclockwise (CCW), Other].

Table A.1: Specification of the 46-Dimensional Atom (Node) Feature Vector (X_i).

Chemical Property	Dimensions	Mathematical / One-Hot Representation & Details
Bond Type	4	One-hot representation of the bond's electronic sharing structure: [Single, Double, Triple, Aromatic].
Conjugation	1	Binary flag indicating if the bond belongs to a continuous, overlapping pi-electron system.
Ring Membership	1	Binary flag indicating if the bond is closed within a ring structure.
Stereochemical Config	4	One-hot representation of spatial configuration around double bonds: [None, Stereo-Z, Stereo-E, Other].

Table A.2: Specification of the 10-Dimensional Bond (Edge) Feature Vector (E_{ij}).

A.1 Symmetry Collapse: Representation-Theoretic Derivation

This appendix provides the full representation-theoretic argument for the expressivity limitation of linear, permutation-invariant readouts at the mixture level, summarised in Section 2 of the main text.

Consider a molecular graph G whose set of node embeddings M occupies a d -dimensional representation space $\mathcal{U}$. The structural symmetries of the graph are captured by its automorphism group $\mathcal{H}$, which collects all permutations of the node indices that leave the graph structure unchanged. When a linear, $\mathcal{H}$ -invariant readout function f is applied to aggregate these embeddings, representation theory requires that it factor through the group-averaging Reynolds operator p_{avg} , defined as [44]:

$$p_{\text{avg}} = \frac{1}{|\mathcal{H}|} \sum_{h \in \mathcal{H}} \rho(h). \quad (\text{A.1})$$

The Reynolds operator projects the full representation space $\mathcal{U}$ onto its trivial, symmetry-fixed subspace $\mathcal{U}_1$. As a consequence, any linear $\mathcal{H}$ -invariant map decomposes into a downstream linear map composed with this group-averaging projection [44]:

$$f = f' \circ p_{\text{avg}}. \quad (\text{A.2})$$

This factorization establishes that a linear readout can only operate on the information that survives in $\mathcal{U}_1$. The non-trivial, symmetry-sensitive components $\mathcal{U}_2 \oplus \dots \oplus \mathcal{U}_m$, which encode the node-to-node variation responsible for structural differences between molecules, are irreversibly discarded by the projection [44]. As a result, even when the Level 1 GAT encoder produces embeddings that are genuinely sensitive to structural differences between molecules, that information does not survive a linear mixture-level readout. Two distinct molecular combinations that share the same element-wise sum or mean are mapped to identical blend vectors and become indistinguishable to any downstream classifier, regardless of how expressive the encoder is. This expressivity ceiling motivates replacing linear pooling at the mixture level with an attention-based readout that operates on the full representation space [41, 44].

A.2 Dataset Profiles and Canonical Vocabulary

A.2.1 Single-Molecule Harmonized Dataset Sample

Table A.3 provides representative single-molecule entries resulting from the consolidation of the GoodScents and Leffingwell databases under RDKit organic sanitization and canonicalization.

Table A.3: Representative sample of harmonized single odorants.

CID	Common Name	Canonical SMILES	Harmonized Descriptors
1183	Vanillin	<chem>COC1CC(C=O)CCC1O</chem>	chocolate, creamy, sweet, vanilla
1254	Menthol	<chem>CC1CCC(C(C)C)C(O)C1</chem>	cooling, mentholic, minty, woody
3314	Eugenol	<chem>C=CCc1ccc(O)c(OC)c1</chem>	cinnamon, clove, phenolic, spicy, woody
10582	Myrtenol	<chem>CC1(C)C2CC=C(CO)C1C2</chem>	balsamic, camphoreous, herbal, pine
31276	Isoamyl acetate	<chem>CC(=O)OCCC(C)C</chem>	banana, estery, fruity, pear, sweet
87630	2-Decen-1-ol	<chem>CCCCCCC=CCO</chem>	citrus, fatty, floral, rose, waxy
637511	Cinnamaldehyde	<chem>O=C/C=C/c1ccccc1</chem>	candy, cinnamon, spicy, warm

A.2.2 Pairwise Blend Mixture Sample

Table A.4 presents sample binary mixtures from the pairwise blend benchmark dataset compiled by Sisson et al. [41], illustrating the constituent molecules and their emergent olfactory descriptors.

Table A.4: Sample pairwise odor mixtures from the Sisson et al. benchmark with emergent descriptors.

Blend	Component 1 (CID)	Component 2 (CID)	Emergent Notes
B-01	Acetaldehyde (177)	2-Heptanol (10976)	citrus, fruity, pungent
B-02	Acetoin (179)	Bornylcyclohexanol (111394)	buttery, creamy, woody
B-03	2-Oxobutanoic acid (58)	Methyl dihydrojasmonate (102861)	brown, caramellic, floral
B-04	2-Oxobutanoic acid (58)	γ -Undecalactone (7714)	brown, caramellic, fruity
B-05	Pentopyranose (229)	Perillaldehyde (16441)	herbal, spicy
B-06	2-Oxobutanoic acid (58)	Butyric acid (264)	cheesy, sour
B-07	Cinnamaldehyde (637511)	Eugenol (3314)	spicy

A.2.3 Standardized 104-Odor Descriptor Vocabulary Gate

Table A.5 enumerates the complete 104-descriptor standardized vocabulary gate enforced during Phase 1 preprocessing.

Table A.5: The 104 standardized odor descriptors (alphabetical).

1. acetic	27. clean	53. honey	79. potato
2. acidic	28. cocoa	54. jammy	80. powdery
3. alcoholic	29. coconut	55. juicy	81. pungent
4. aldehydic	30. coffee	56. lactonic	82. roasted
5. alliaceous	31. cooling	57. leathery	83. rooty
6. amber	32. corn	58. licorice	84. rummy
7. ammoniacal	33. coumarinic	59. malty	85. salty
8. animal	34. creamy	60. marine	86. smoky
9. anise	35. dairy	61. meaty	87. soapy
10. aromatic	36. dusty	62. medicinal	88. solvent
11. balsamic	37. earthy	63. melon	89. sour
12. berry	38. eggy	64. mentholic	90. spicy
13. bitter	39. estery	65. minty	91. sulfurous
14. bready	40. ethereal	66. moldy	92. sweet
15. brown	41. fatty	67. mossy	93. tarragon
16. burnt	42. fermented	68. mushroom	94. thujonic
17. buttery	43. fishy	69. musk	95. toasted
18. cabbage	44. floral	70. mustard	96. tobacco
19. camphoreous	45. fresh	71. musty	97. tomato
20. caramellic	46. fruity	72. nutty	98. tonka
21. celery	47. fungal	73. oily	99. tropical
22. cheesy	48. fusel	74. onion	100. vanilla
23. chemical	49. garlic	75. orris	101. vegetable
24. cherry	50. green	76. peach	102. waxy
25. chocolate	51. hay	77. phenolic	103. winy
26. citrus	52. herbal	78. pine	104. woody

REFERENCES

- [1] R. ACHEBOUCHE, A. TROMELIN, K. AUDOUZE, AND O. TABOUREAU, *Application of artificial intelligence to decode the relationships between smell, olfactory receptors and small molecules*, Scientific Reports, 12 (2022), p. 18817.
- [2] R. L. ALMEIDA, V. G. MALTAROLLO, AND F. G. F. COELHO, *Overcoming class imbalance in drug discovery problems: Graph neural networks and balancing approaches*, Journal of Molecular Graphics and Modelling, 126 (2024), p. 108627.
- [3] D. AMETA, S. KUMAR, R. MISHRA, L. BEHERA, A. CHAKRABORTY, AND T. SANDHAN, *Odor classification: Exploring feature performance and imbalanced data learning techniques*, PLOS ONE, 20 (2025), p. e0322514.
- [4] A. R. N. AOUICHAOU, F. FAN, S. S. MANSOURI, J. ABILDSKOV, AND G. SIN, *Combining group-contribution concept and graph neural networks toward interpretable molecular property models*, Journal of Chemical Information and Modeling, 63 (2023), pp. 725–744.
- [5] J. B. CASTRO, T. J. GOULD, R. PELLEGRINO, Z. LIANG, L. A. COLEMAN, F. PATEL, D. S. WALLACE, T. BHATNAGAR, J. D. MAINLAND, AND R. C. GERKIN, *Pyrfume: A window to the world’s olfactory data*, bioRxiv, (2022).
- [6] C. CHEN, M. SHENG, H. YUAN, X. PAN, H. TIAN, AND X. LOU, *Perceptual interaction of aroma compounds and their effects on the enhancement of flavor quality in foods: a review*, Trends in Food Science & Technology, 165 (2025), p. 105315.
- [7] S. CHITHRANANDA, J. AMORES, AND K. K. YANG, *Mapping the combinatorial coding between olfactory receptors and perception with deep learning*, bioRxiv, (2026).
- [8] G. COUREAUD, T. THOMAS-DANGUIN, J.-C. SANDOZ, AND D. A. WILSON, *Biological constraints on configural odour mixture perception*, J Exp Biol, 225 (2022).
- [9] R. DATTA, J. NISTANE, A. SOSE, H. SAHU, AND R. RAMPRASAD, *Machine learning for green solvents: Assessment, selection and substitution*, Advanced Science, (2025).
- [10] H. FOOLADI, T. N. L. VU, M. MATHEA, AND J. KIRCHMAIR, *Evaluating machine learning models for molecular property prediction: Performance and robustness on out-of-distribution data*, Journal of Chemical Information and Modeling, 65 (2025), pp. 9871–9891.
- [11] Y. FUKUTANI, M. ABE, H. SAITO, R. EGUCHI, T. TAZAWA, C. A. DE MARCH, M. YOHDA, AND H. MATSUNAMI, *Antagonistic interactions between odorants alter human odor perception*, Current Biology, 33 (2023), pp. 2235–2245.e4. doi: 10.1016/j.cub.2023.04.072.

- [12] D. GASSER, S. SINGH, J. FREIHERR, AND A. GRASSKAMP, *Owsum: algorithmic odor prediction and insight into structure-odor relationships*, Journal of Cheminformatics, 15 (2023).
- [13] Q. GUO, S. HERNANDEZ-HERNANDEZ, AND P. J. BALLESTER, *Umap-based clustering split for rigorous evaluation of ai models for virtual screening on cancer cell lines**, Journal of Cheminformatics, 17 (2025), p. 94.
- [14] R. HAGAN, D. MOORE, F. MCGLONE, AND S. C. WALKER, *Olfactory scene analysis: Does analytical visual processing predict superior identification of component odours in a complex mixture?*, Perception, 54 (2025), pp. 408–430. PMID: 40165414.
- [15] S. HAN, H. FU, Y. WU, G. ZHAO, Z. SONG, F. HUANG, Z. ZHANG, S. LIU, AND W. ZHANG, *Himgnn: a novel hierarchical molecular graph representation learning framework for property prediction*, Briefings in Bioinformatics, 24 (2023), p. bbad305.
- [16] F. HATAMI AND M. MORADI, *Asymmetric focal loss improves graph neural network prediction of drug-drug interactions*, 2026.
- [17] Y. P. HENG, H. Y. LEE, J. W. CHONG, R. R. TAN, K. B. AVISO, AND N. G. CHEM-MANGATTUVALAPPIL, *Incorporating machine learning in computer-aided molecular design for fragrance molecules*, Processes, 10 (2022).
- [18] M. HLADIŠ, M. LALIS, S. FIORUCCI, AND J. TOPIN, *Matching receptor to odorant with protein language and graph neural networks*, in The Eleventh International Conference on Learning Representations, 2023.
- [19] B. HU, H. ZHENG, H. LIANG, H. LV, Q. XIAO, Y. JIANG, Q. LU, Y. NIU, H. WANG, AND Z. XIAO, *Elucidating odorant masking effects in food matrices through olfactory receptor-based profiling*, bioRxiv, (2026).
- [20] J. HUANG, J. LIN, R. YUENG, S. WU, L. SOLLA, AND T. ACREE, *Masking effects on iso-valeric acid recognition by sub-threshold odor mixture*, Chemical Senses, 48 (2023), p. bjad047.
- [21] R. JOERES, D. B. BLUMENTHAL, AND O. V. KALININA, *Data splitting to avoid information leakage with datasail*, Nature Communications, 16 (2025), p. 3337.
- [22] A. KENGKANNA AND M. OHUE, *Enhancing property and activity prediction and interpretation using multiple molecular graph representations with mmgx*, Communications Chemistry, 7 (2024), p. 74.
- [23] M. LALIS, M. HLADIŠ, S. A. KHALIL, L. BRIAND, S. FIORUCCI, AND J. TOPIN, *M2or: a database of olfactory receptor-odorant pairs for understanding the molecular mechanisms of olfaction*, Nucleic Acids Research, 52 (2024), pp. D1370–D1379.
- [24] B. K. LEE, E. J. MAYHEW, B. SANCHEZ-LENGELING, J. N. WEI, W. W. QIAN, K. A. LITTLE, M. ANDRES, B. B. NGUYEN, T. MOLOY, J. YASONIK, J. K. PARKER, R. C. GERKIN, J. D. MAINLAND, AND A. B. WILTSCHKO, *A principal odor map unifies diverse tasks in olfactory perception*, Science, 381 (2023), pp. 999–1006.
- [25] J. LEE, Y. LEE, J. KIM, A. R. KOSIOREK, S. CHOI, AND Y. W. TEH, *Set transformer: A framework for attention-based permutation-invariant neural networks*, 2019.

- [26] Q. LI, S. LI, J. LUO, AND H. YUAN, *Predictive models of odor contribution and thresholds for volatiles in identification of novel crop aroma compounds*, *Metabolites*, 15 (2025).
- [27] S. LI, X. XIA, S. GE, AND T. LIU, *Selection-based pseudo-labeling for multi-label learning with noisy labels: Estimating the noise transition*, in *Advances in Neural Information Processing Systems*, 2022.
- [28] K. LÓPEZ-PÉREZ, J. F. AVELLANEDA-TAMAYO, L. CHEN, E. LÓPEZ-LÓPEZ, K. E. JUÁREZ-MERCADO, J. L. MEDINA-FRANCO, AND R. A. MIRANDA-QUINTANA, *Molecular similarity: Theory, applications, and perspectives*, *Artificial Intelligence Chemistry*, 2 (2024), p. 100077.
- [29] E. MAKRI, N. NAKIS, L. SISSON, G. MINSKY, L. TASSIULAS, V. SATARIFARD, AND N. A. CHRISTAKIS, *Benchmark for assessing olfactory perception of large language models*, 2026.
- [30] C. PATINHA CALDEIRA, R. FARCAL, C. MORETTI, L. MANCINI, H. RAUSCHER, K. RASMUSSEN, J. RIEGO SINTES, AND S. SALA, *Safe and sustainable by design chemicals and materials: Review of safety and sustainability dimensions, aspects, methods, indicators, and tools*, EUR 30991 EN, Joint Research Centre (JRC), European Commission, Luxembourg, 2022.
- [31] A. PAUL, *Dextrosinistral reading of smiles notation: Investigation into origin of non-sense code from string manipulations*, *Digital Chemical Engineering*, 15 (2025), p. 100222.
- [32] R. PELLEGRINO, E. J. MAYHEW, J. MARGOLIS, M. ANDRES, A. B. WILTSCHKO, R. C. GERKIN, AND J. D. MAINLAND, *Odors smell like their components: A linear framework for predicting olfactory mixture perception*, *bioRxiv*, (2026).
- [33] S. QIN, S. JIANG, J. LI, P. BALAPRAKASH, R. C. VAN LEHN, AND V. M. ZAVALA, *Capturing molecular interactions in graph neural networks: a case study in multi-component phase equilibrium*, *Digital Discovery*, 2 (2023), pp. 138–151.
- [34] O. RAINIO, J. TEUHO, AND R. KLÉN, *Evaluation metrics and statistical tests for machine learning*, *Scientific Reports*, 14 (2024), p. 6086.
- [35] B. C. L. RODRIGUES, V. V. SANTANA, S. MURINS, AND I. B. R. NOGUEIRA, *Molecule generation and optimization for efficient fragrance creation*, *Industrial & Engineering Chemistry Research*, 63 (2024), pp. 14480–14494.
- [36] S. ROMAGNY, G. COUREAUD, AND T. THOMAS-DANGUIN, *Developmental switch from elemental to configural processing of odor mixtures in humans*, *Developmental Psychobiology*, 68 (2026), p. e70188.
- [37] K. SAINI AND V. RAMANATHAN, *Predicting odor from molecular structure: a multi-label classification approach*, *Scientific Reports*, 12 (2022), p. 13863.
- [38] A. SAJAN, S. SLUIS, R. HAYDARLOU, S. ABELN, P. LISENA, R. TRONCY, C. VERBEEK, I. LEEMANS, AND H. MOUHIB, *Hierarchies of smell: Structuring the molecular odor space using semantic taxonomies and machine learning*, *Chemical Senses*, 51 (2026), p. bjag020.
- [39] E. I. SANCHEZ MEDINA, S. LINKE, M. STOLL, AND K. SUNDMACHER, *Gibbs–helmholtz graph neural network: capturing the temperature dependency of activity coefficients at infinite dilution*, *Digital Discovery*, 2 (2023), pp. 781–798.

- [40] H. SHIMAKAWA, A. KUMADA, AND M. SATO, *Extrapolative prediction of small-data molecular property using quantum mechanics-assisted machine learning*, npj Computational Materials, 10 (2024), p. 11.
- [41] L. SISSON, A. A. BARSAINYAN, M. SHARMA, AND R. KUMAR, *Deep learning for odor prediction on aroma-chemical blends*, ACS Omega, 10 (2025), pp. 8980–8992.
- [42] S. STESHIN, *Lo-hi: Practical ml drug discovery benchmark*, 2023.
- [43] J. SUH, Y. HONG, AND C. PARK, *A comparative study of machine learning models on molecular fingerprints for odor decoding*, Communications Chemistry, 8 (2025), p. 278.
- [44] M. TALHI, A. WOLF, AND A. MONOD, *Breaking symmetry bottlenecks in gnn readouts*, 2026.
- [45] A. N. TAREKEGN, M. GIACOBINI, AND K. MICHALAK, *A review of methods for imbalanced multi-label classification*, Pattern Recognition, 118 (2021), p. 107965.
- [46] G. TOM, C. T. SER, E. RAJAONSON, S. LO, H. S. PARK, B. LEE, AND B. SANCHEZ-LENGELING, *Does this smell the same? learning representations of olfactory mixtures using inductive biases*, Machine Learning: Science and Technology, 6 (2025).
- [47] J. WANG, Q. ZHANG, W. FAN, Q. SHI, J. MAO, J. XIE, G. CHAI, AND C. ZHANG, *Deciphering olfactory receptor binding mechanisms: a structural and dynamic perspective on olfactory receptors*, Frontiers in Molecular Biosciences, Volume 11 - 2024 (2025).
- [48] T. WEN, X. CAI, AND J. LI, *Graph neural networks vs. traditional qsar: A comprehensive comparison for multi-label molecular odor prediction*, Molecules, 30 (2025).
- [49] D. S. WIGH, J. M. GOODMAN, AND A. A. LAPKIN, *A review of molecular representation in the age of machine learning*, WIREs Computational Molecular Science, 12 (2022), p. e1603.
- [50] Y.-X. WU, X. WANG, A. ZHANG, X. HE, AND T.-S. CHUA, *Discovering invariant rationales for graph neural networks*, 2022.
- [51] Z. WU, J. WANG, H. DU, D. JIANG, Y. KANG, D. LI, P. PAN, Y. DENG, D. CAO, C.-Y. HSIEH, AND T. HOU, *Chemistry-intuitive explanation of graph neural networks for molecular property prediction with substructure masking*, Nature Communications, 14 (2023), p. 2585.
- [52] H. XIE, J. SUN, Y. SHAO, S. LI, S. HOU, Y. SUN, AND J. WANG, *Molecular odor prediction based on multi-feature graph attention networks*, 2025.
- [53] L. XU, D.-J. ZOU, AND S. FIRESTEIN, *Odor mixtures: A chord with silent notes*, Frontiers in Ecology and Evolution, Volume 11 - 2023 (2023).
- [54] M. ZAHEER, S. KOTTUR, S. RAVANBAKHS, B. POCZOS, R. SALAKHUTDINOV, AND A. SMOLA, *Deep sets*, 2018.
- [55] M. ZHANG, Y. HIKI, A. FUNAHASHI, AND T. J. KOBAYASHI, *A deep position-encoding model for predicting olfactory perception from molecular structures and electrostatics*, NPJ systems biology and applications, 10 (2024), p. 76.
- [56] W. ZHU, Y. ZHANG, D. ZHAO, J. XU, AND L. WANG, *Hignn: Hierarchical informative graph neural networks for molecular property prediction equipped with feature-wise attention*, 2022.